CG1111A

Engineering Principles and Practice I

A-maze-ing Race Project

August 2025 Semester 1

Group Number: B01

Section: 4

Team Number: 1

Submitted by

Lee Pei Ling, A0323785W

Gavin Lim, A0324539Y

Lim Sin, A0321868W

Lok Wei Xiong, A0324469W

Table of Contents

1. Introduction

1.1 Project Details

The goal of the project is to design and program a mBot capable of navigating through a maze in the shortest time possible while completing various waypoint challenges. These challenges involve detecting coloured waypoints and executing the correct turning manoeuvres based on the decoded colour signals. The robot integrates multiple sensors, including an ultrasonic sensor, an infrared (IR) sensor, together with a line sensor and a colour detector. This allows the mBot to detect maze walls to align itself, and perform corresponding manoeuvres based on the detected coloured waypoints.

The project combined both hardware design and software programming aspects which allowed our team to gain practical experience in circuit design, calibration, code debugging and algorithm design.

In this report, we will discuss the overall system design, hardware implementation, software algorithms, sensor calibration methods, testing procedures, and team contributions.

1.2 Overview of Design

Table A shows the overall design of our mBot. The ultrasonic sensor was mounted on the left and the IR sensor was mounted on the right. They were positioned on opposite sides to allow accurate wall detection and alignment within the maze. The colour detector, consisting of RGB LEDs and a light-dependent resistor (LDR), was mounted beneath the chassis to detect coloured waypoints on the maze floor. The line sensor was mounted close to the front of the chassis. The components and wirings were arranged neatly and compact to prevent any brushing of wires against the maze walls. To reduce ambient light from entering the colour detector, we designed a double-layered skirting around the colour detector circuit to improve reading consistency.

Table B shows a close-up image of both the colour detector circuit and the IR sensor circuit. This provides a clearer picture of how we connected our circuit as described in Section 2.4. We ensured that all wires were cut neatly and secured tightly to ensure that they do not come loose and obstruct the LDR or IR sensor.

1.3 GitHub Documentation

An overview of our project documentation and demonstration can be found using this link: https://github.com/gavinlimsh/CG1111A-A-maze-ing-Race

Table A: Images of the mBot

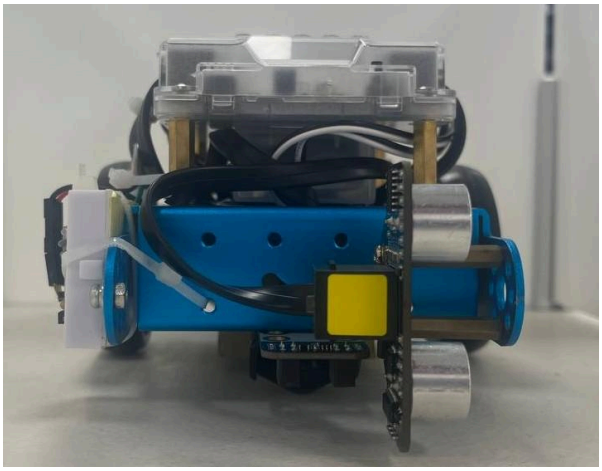
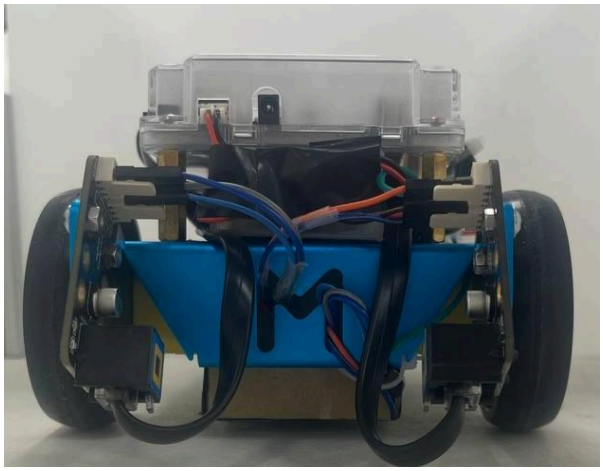
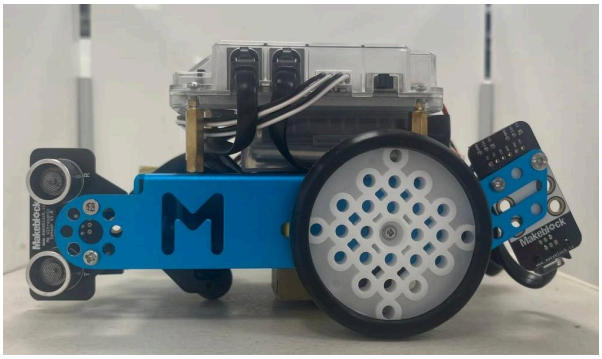
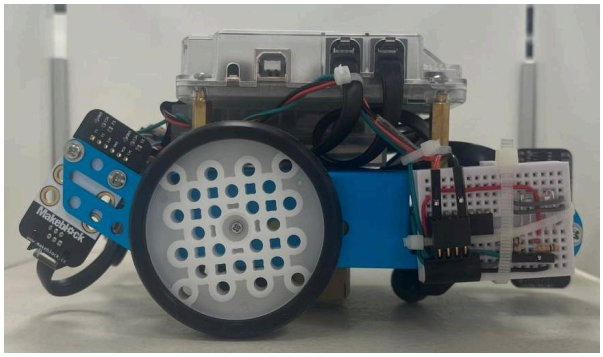
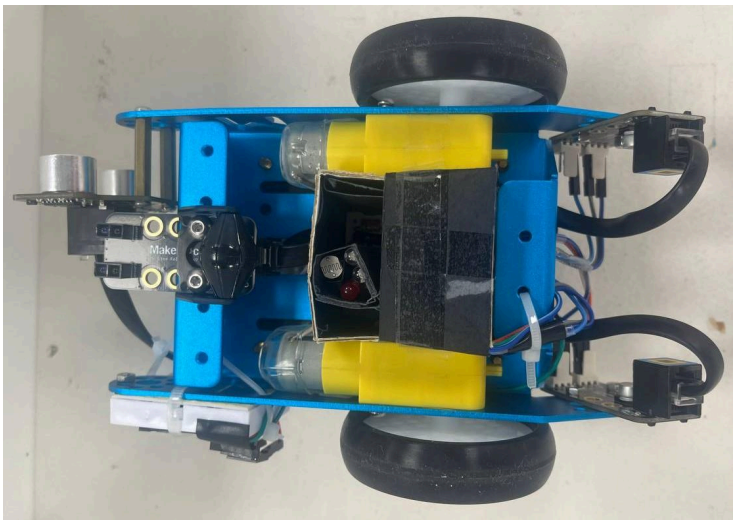
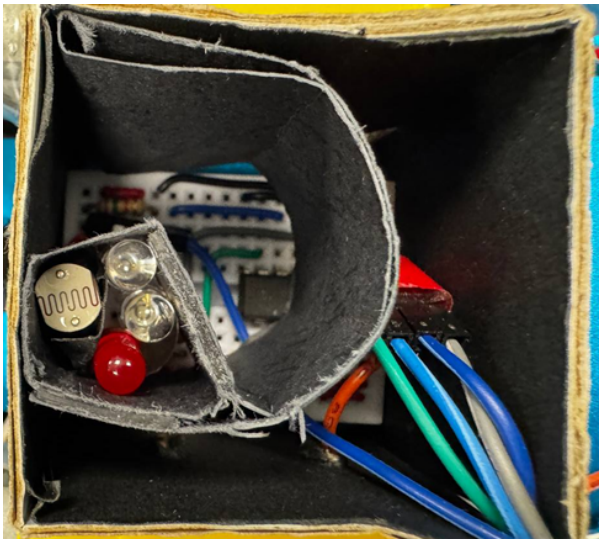
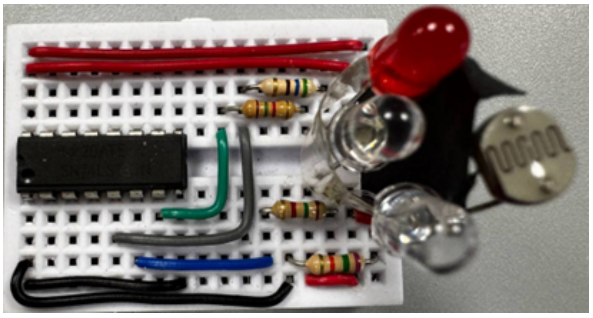
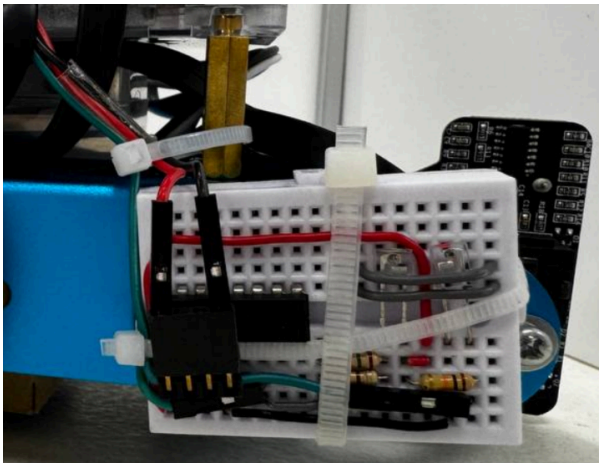
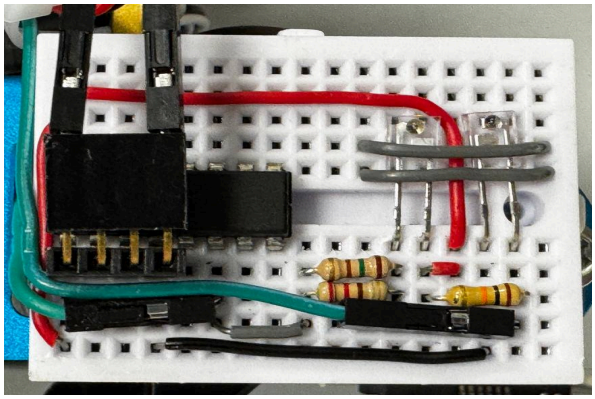
Front View	Back View
	
Left View	**Right View**
	
Bottom View	
	

Table B: Images of the IR Sensor and Colour Detector Circuits

Colour Detector	Zoomed-in image
	
IR Sensor	**Zoomed-in image**
	

2. Hardware Design

2.1 Equipment List

Table C: List of Equipment Used

Item Description	Quantity
mBot Chassis	1
DC Motor and Wheels	2
mCore	1
Me Line-follower Sensor	1
RJ25 Adaptor	2
Me Ultrasonics Sensor	1
LTR 301 (IR Detector) and LTR 302 (IR Emitter)	1
Light Dependent Resistor (LDR)	1
Light-emitting Diodes (Red, Green, Blue)	3
HD74LS139P 2-to-4 decoder IC chip	1
L293D H-bridge Motor Driver	1
Resistors of varying resistance	7

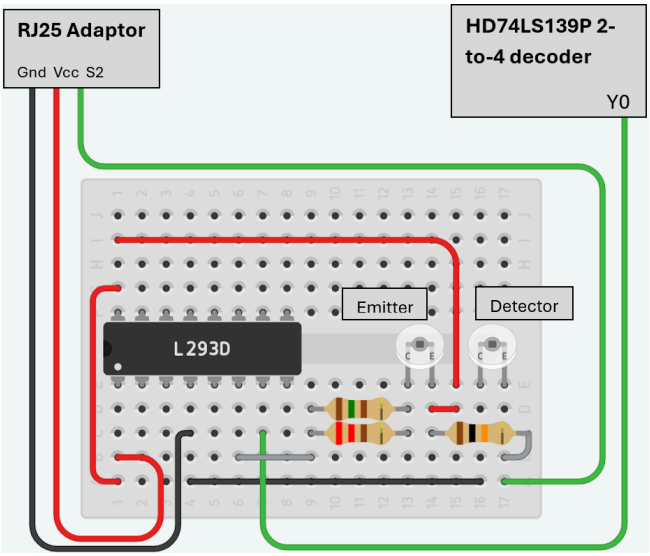
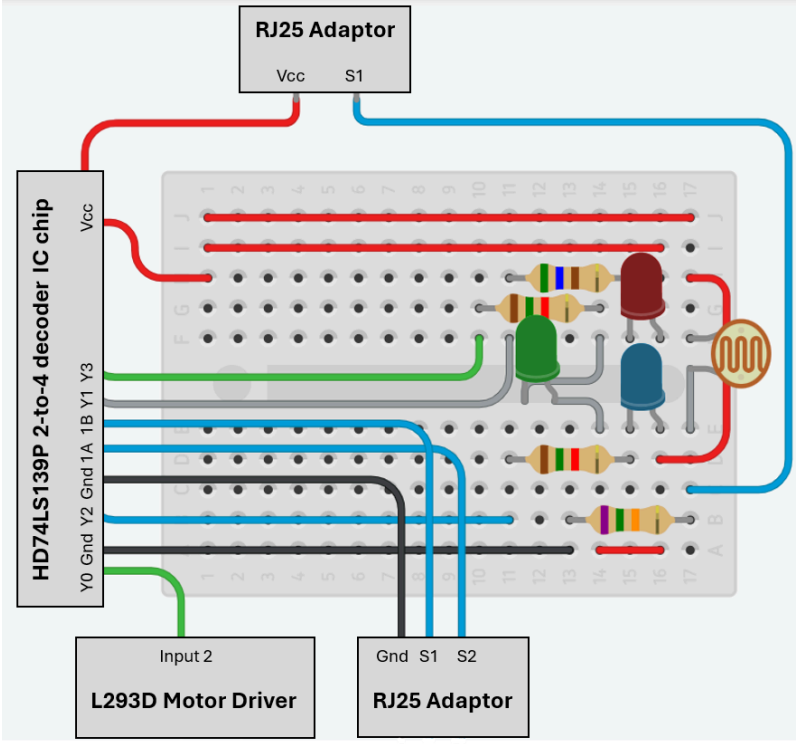
2.2 mBot Connections

Table D: Sensor and Motor Connections

Sensor Connections		Motor Connections	
Sensor	**Port**	**Side**	**Port**
Ultrasonic Sensor	2	Left	M1
Line Follower	3	Right	M2

2.3 TinkerCAD Circuit Designs

Table E: Overview of the IR Sensor and Colour Detector Circuits

IR Sensor Circuit

Colour Detector Circuit


2.4 Overall Schematic Design

Figure 1 shows the overall connections between the different RJ25 Adaptors, the ultrasonic sensor, line sensor, colour detector circuit and IR circuit. More detailed explanations of the different connections are described in the subsequent sub-sections.

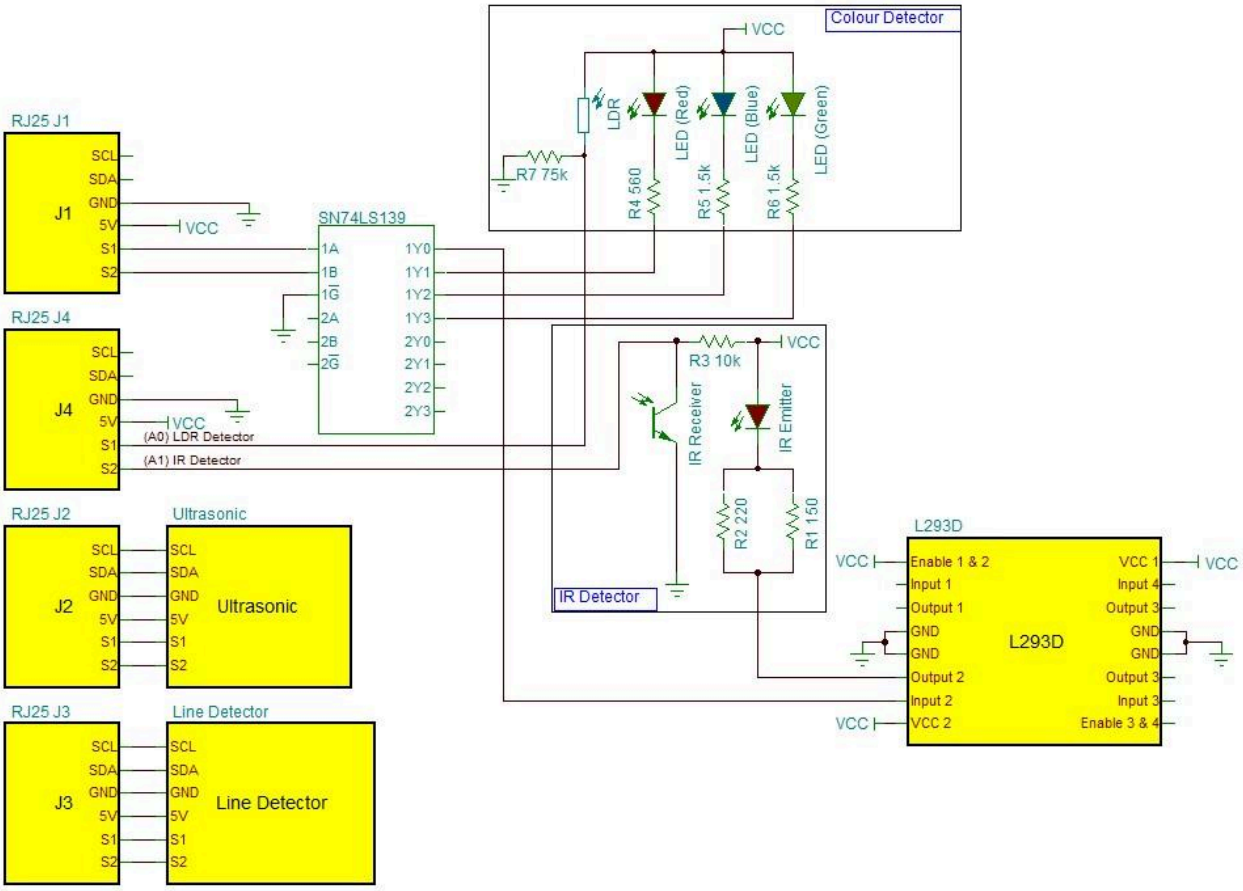


Figure 1: Overall Schematic Diagram

2.5 Colour Detector

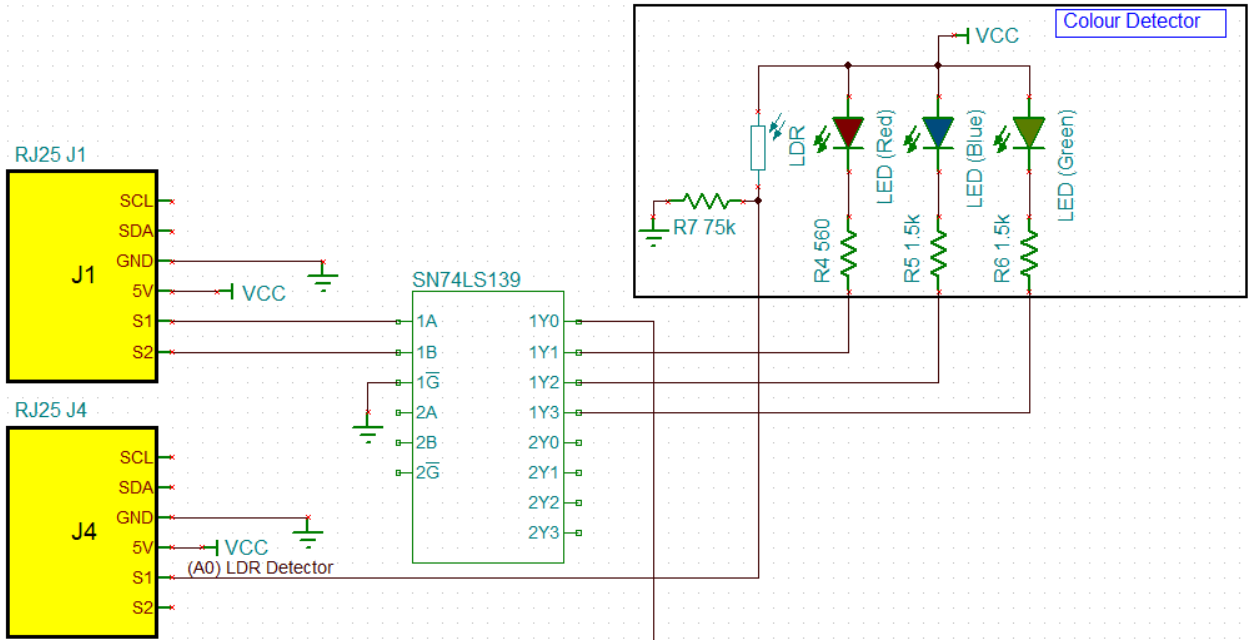


Figure 2: Colour Detector Schematic

The colour detector consists of 3 RGB LEDs which are used to illuminate the coloured paper under the sensor:

1) Multicomp PRO 5mm Round LED (Red)
2) Cree® 5-mm Round LED (Blue and Green)

Inputs			Outputs			
Enable	Select					
G	B	A	Y_0	Y_1	Y_2	Y_3
H	X	X	H	H	H	H
L	L	L	L	H	H	H
L	L	H	H	L	H	H
L	H	L	H	H	L	H
L	H	H	H	H	H	L

H ; high level, L ; low level, X ; irrelevant

Figure 3: Truth Table for HD74LS139 - Dual 2-line-to-4-line Decoders / Demultiplexers

From Figure 2, the HD74LS139 decoder's enable pin (G) is connected to GND, which permanently enables the decoder, allowing it to operate according to the truth table highlighted in Figure 3.

The decoder select inputs (A and B) are connected to the S1 and S2 pins of the MakeBlock RJ25 adaptor, which is then connected to port J1 of the mCore board. By varying the select inputs using digitalWrite(), the decoder is able to control the lighting of each LED.

For example, when both select inputs are set to HIGH, the data output Y_3 will be pulled to LOW (ideally 0V), while the other data outputs (Y_0 - Y_2) remain HIGH. As a result, the green LED will turn on since there is a voltage difference. By varying the states of the select inputs, the decoder is able to control when each LED lights up.

To detect the intensity of the colours that is reflected back from the surface, a LDR is used. When a specific LED is activated by the decoder through the select pins, it illuminates the coloured paper underneath the mBot. The LDR will then detect the light that is reflected back from the coloured surface. The voltage across the 75kΩ resistor is then measured by S1 of the RJ25 adaptor (J4) using analogRead(), which maps the analog voltage reading from 0 to 1023.

2.5.1 Minimum Resistor Values for Individual LED

Recommended Operating Conditions

Item	Symbol	Min	Typ	Max	Unit
Supply voltage	V_{CC}	4.75	5.00	5.25	V
Output current	I_{OH}	—	—	–400	μA
	I_{OL}	—	—	8	mA
Operating temperature	Topr	–20	25	75	°C

Figure 4: Datasheet for HD74LS139 - Dual 2-line-to-4-line Decoders / Demultiplexers

In Figure 2, the data outputs (Y_0 - Y_3) of the HD74LS139 decoder acts as current sinks for the 3 different LEDs when the decoder pulls the data output pin to logic LOW based on the truth table as shown in Figure 3.

However, the maximum output current of the decoder is 8mA as shown in Figure 4. If excessive current is supplied to the decoder, the chip's internal protection circuitry will raise the logic LOW's voltage to limit the current to 8mA, which may cause the output voltage to fluctuate unpredictably.

As a result, it is crucial that the biasing resistors that are used for the LEDs limit the total current to be smaller than 8mA.

Electrical / Optical Characteristics at Ta = 25°C

Parameter	Symbol	Min.	Typ.	Max.	Unit	Condition
Luminous Intensity	IV	70	100	130	mcd	IF = 20mA
Viewing Angle	2θ1/2		45		deg	
Peak Emission Wavelength	λp		650		nm	
Dominant Wavelength	λD	635	640	645	nm	
Spectral Line Half-Width	Δλ		20		nm	
Forward Voltage	VF	1.8	1.9	2.2	V	
Reverse Current	Ir	-	-	10	uA	VR=5V

Figure 5: Datasheet for Multicomp PRO 5mm Round LED (Red)

Characteristics	Color		Symbol	Condition	Unit	Minimum	Typical	Maximum
Forward Voltage	Blue/Green		V_F	I_F = 20 mA	V		3.2	4.0
Reverse Current	Blue/Green		I_R	V_R = 5 V	µA			100
Dominant Wavelength	Blue		λ_D	I_F = 20 mA	nm	465	470	480
	Green		λ_D	I_F = 20 mA	nm	520	527	535
Luminous Intensity	Blue	C503B-BCS/BCN (30 degree)	I_V	I_F = 20 mA	mcd	2130	4800	
	Green	C503B-GCS/GCN (30 degree)	I_V	I_F = 20 mA	mcd	5860	20000	
50% Power Angle	C503B-BCS/BCN/GCS/GCN		2θ½	I_F = 20 mA	deg		30	

Figure 6: Datasheet for Cree® 5-mm Round LED (Blue and Green)

Based on the datasheets in Figure 5 and 6, the forward voltage for the Red LED ranges from 1.8V to 2.2V whereas the forward voltage for the Blue and Green LED ranges from 3.2V to 4.0V.

In our calculations, the forward voltage values that we used were 1.9V (Red LED) and 3.2V (Blue and Green LED). We also assumed that the output LOW of the HD74LS139 decoder is 0V and the Vcc from the mCore is 5V.

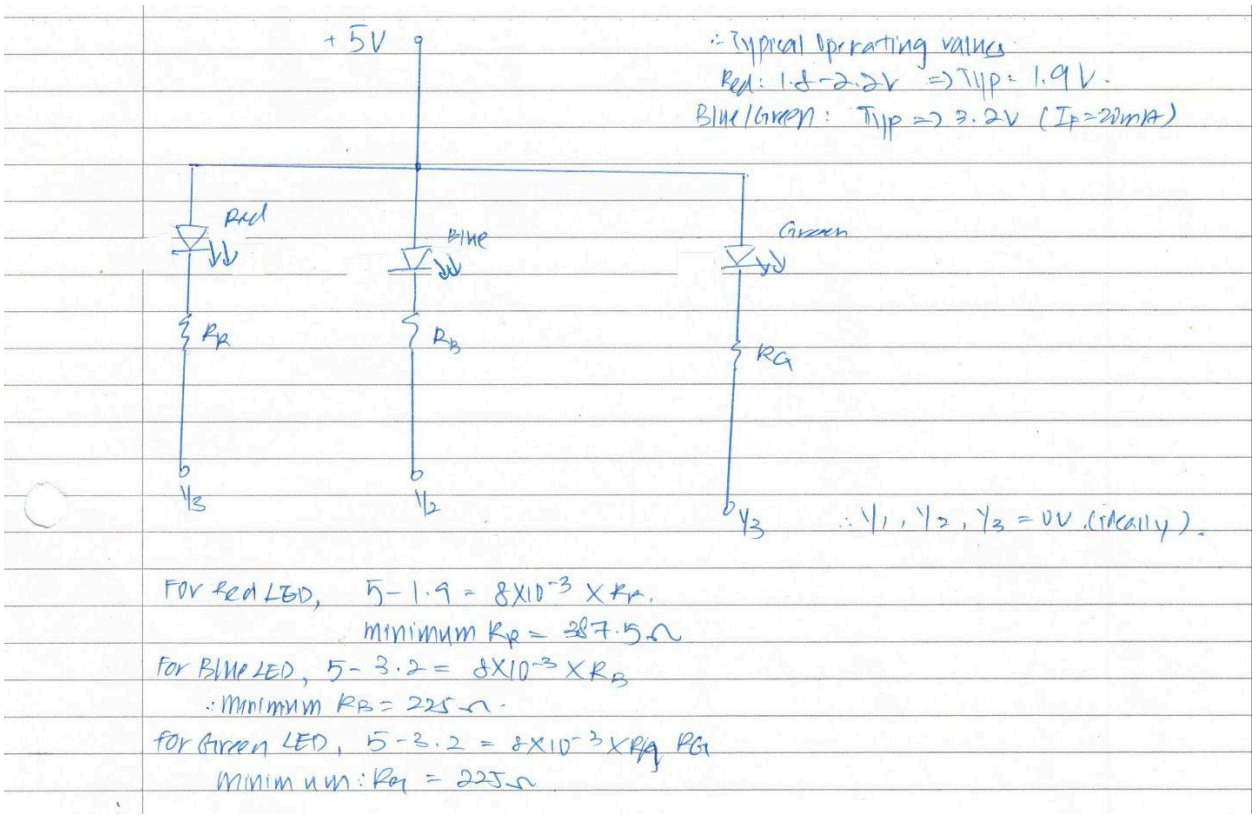


Figure 7: Minimum biasing resistor calculations (LED)

Based on the calculations shown in Figure 7, the minimum biasing resistor that we can use is 387.5Ω (Red LED) and 225Ω (Blue and Green LED). As a result, we only can use resistors above this threshold to ensure that the current into the decoder is smaller than 8mA.

2.5.2 Finalised Resistor Values for all LEDs

1.0	10	100	1.0 k	10 k	100 k	1.0 M
1.1	11	110	1.1 k	11 k	110 k	1.1 M
1.2	12	120	1.2 k	12 k	120 k	1.2 M
1.3	13	130	1.3 k	13 k	130 k	1.3 M
1.5	15	150	1.5 k	15 k	150 k	1.5 M
1.6	16	160	1.6 k	16 k	160 k	1.6 M
1.8	18	180	1.8 k	18 k	180 k	1.8 M
2.0	20	200	2.0 k	20 k	200 k	2.0 M
2.2	22	220	2.2 k	22 k	220 k	2.2 M
2.4	24	240	2.4 k	24 k	240 k	2.4 M
2.7	27	270	2.7 k	27 k	270 k	2.7 M
3.0	30	300	3.0 k	30 k	300 k	3.0 M
3.3	33	330	3.3 k	33 k	330 k	3.3 M
3.6	36	360	3.6 k	36 k	360 k	3.6 M
3.9	39	390	3.9 k	39 k	390 k	3.9 M
4.3	43	430	4.3 k	43 k	430 k	4.3 M
4.7	47	470	4.7 k	47 k	470 k	4.7 M
5.1	51	510	5.1 k	51 k	510 k	5.1 M
5.6	56	560	5.6 k	56 k	560 k	5.6 M
6.2	62	620	6.2 k	62 k	620 k	6.2 M
6.8	68	680	6.8 k	68 k	680 k	6.8 M
7.5	75	750	7.5 k	75 k	750 k	7.5 M
8.2	82	820	8.2 k	82 k	820 k	8.2 M
9.1	91	910	9.1 k	91 k	910 k	9.1 M

Figure 8: Standard Resistor Table (±5%) (The Open University, n.d.)

Parameter	Symbol	Min.	Typ.	Max.	Unit
Luminous Intensity	IV	70	100	130	mcd

Figure 9: Luminous Intensity for Multicomp PRO 5mm Round LED (Red)

Characteristics	Color		Symbol	Condition	Unit	Minimum	Typical	Maximum
Luminous Intensity	Blue	C503B-BCS/BCN (30 degree)	I_v	I_F = 20 mA	mcd	2130	4800	
	Green	C503B-GCS/GCN (30 degree)	I_v	I_F = 20 mA	mcd	5860	20000	

Figure 10: Luminous Intensity for Cree® 5-mm Round LED (Blue and Green)

Table F: Theoretical vs Implemented Biasing Resistor Values

LED Colour	Minimum	Actual
Red	387.5Ω	560Ω
Blue	225Ω	1500Ω
Green	225Ω	1500Ω

Based on our observations, we realized that the blue and green LEDs were significantly brighter than the red LED at equal operating currents. This brightness difference is evident in Figures 9 and 10, which compare the luminous intensity specifications of the two LED models. The Cree®

blue and green LEDs have typical luminous intensities ranging from 4,800 to 20,000 mcd whereas the Multicomp PRO red LEDs range from 70 to 130 mcd.

As shown in Table F, the biasing resistors for the green and blue LEDs (1500Ω each) are significantly higher than the resistor for the red LED (560Ω). By using a higher biasing resistor, the forward current through the blue and green LEDs is significantly reduced, thereby reducing its brightness. As a result, the LDR sensor readings will not be oversaturated, allowing it to distinguish between spectrally similar colours such as red and orange.

With this resistor configuration (560Ω, 1500Ω, 1500Ω), we are able to obtain a calibrated greydiff[] of 390.00 (Red), 179.00 (Blue), 218.00 (Green), which represents the ADC dynamic range between black and white. As a result, the colour detecting circuit is able to accurately distinguish between the different colored papers during navigation.

2.5.3 Finalised Resistor Values for LDR

For the LDR voltage divider circuit, a 75kΩ resistor was utilized instead of the 100kΩ resistor that we used in the studio sessions. When we used the 100kΩ resistor, the voltage range between dark and illuminated conditions were insufficient for accurate colour detection. As such, we reduced the resistor to 75kΩ. This configuration enables a greater voltage range without compromising our overall ADC dynamic range, thereby allowing it to distinguish between the different coloured papers.

2.6 IR Sensor

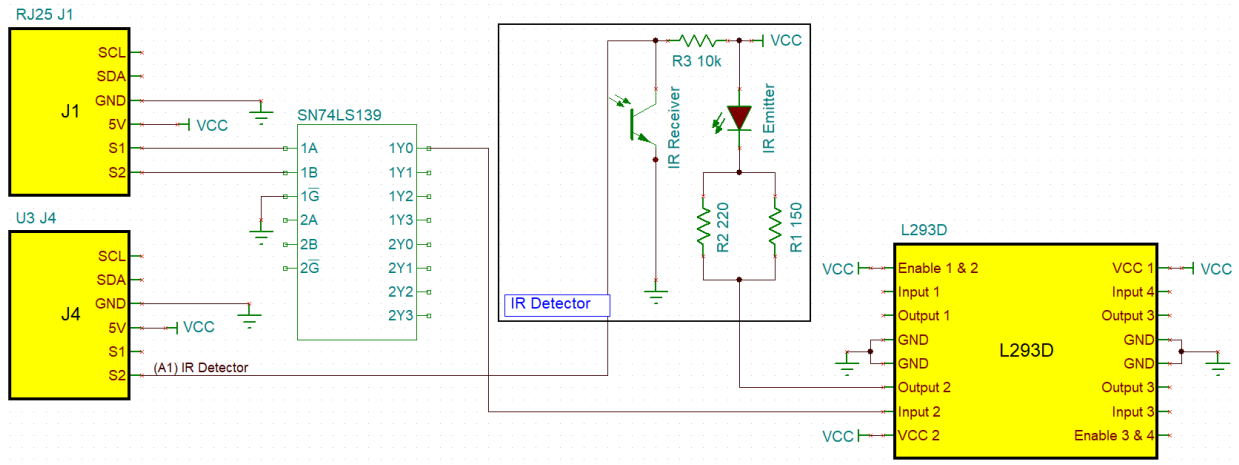


Figure 11: IR Sensor Schematic

The IR Sensor consists of the IR Emitter (LTE-302) and the IR Detector (LTE-301). The IR Sensor works based on the IR Indirect Incidence Principle, where the IR Emitter and Detector must be placed side by side.

Inputs			Outputs			
Enable	Select					
G	B	A	Y_0	Y_1	Y_2	Y_3
H	X	X	H	H	H	H
L	L	L	L	H	H	H
L	L	H	H	L	H	H
L	H	L	H	H	L	H
L	H	H	H	H	H	L

H ; high level, L ; low level, X ; irrelevant

Figure 12: Truth table for HD74LS139 decoder (IR detector mode)

The IR detector will be activated when the select pins to the HD74LS139 decoder are both LOW. Based on the truth table in Figure 12, the data output Y_0 will be pulled to LOW (ideally 0V), while the other data outputs (Y_1 - Y_3) remain HIGH.

Input	Enable (*)	Output
H	H	H
L	H	L
H	L	Z
L	L	Z

Figure 13: Truth table for L293D Channel Driver (Enable HIGH Configuration)

From Figure 11, the data output Y_0 from the decoder is connected to Input 2 of the L293D Channel Driver. As such, when the data output Y_0 is pulled to LOW, input 2 of the L293D will also be LOW.

When the enable pin of the L293D Channel Driver is pulled to HIGH, it will operate according to the truth table highlighted in Figure 3. As such, when the input 2 is LOW, there will be a voltage difference between the IR Emitter and it will emit a 940nm IR light onto the surface. The reflected 940nm IR will turn on the phototransistor in the IR Detector and the voltage across the phototransistor depends on the reflected IR intensity.

The mCore will then determine the distance of the object from the IR by reading the voltage across the IR Detector using analogRead(), which maps the analog voltage reading from 0 to 1023. A lower voltage reading indicates that the object is in close proximity, and vice versa.

2.6.1 Maximum Resistor Values for IR Sensor

PARAMETER	MAXIMUM RATING	UNIT
Power Dissipation	75	mW
Peak Forward Current (300pps, 10 μ s pulse)	1	A
Continuous Forward Current	50	mA
Reverse Voltage	5	V
Operating Temperature Range	-40℃ to + 85℃	
Storage Temperature Range	-55℃ to + 100℃	
Lead Soldering Temperature [1.6mm(.063") From Body]	260℃ for 5 Seconds	

Figure 14: Maximum Forward Current (IR Emitter)

To enhance the IR Sensor detection range, the L293D motor driver chip is used to power the IR Emitter by allowing it to operate at its maximum current level of 50mA as shown in Figure 14 instead of 8mA, which is the maximum output current of the HD74LS139 decoder. This will significantly increase the effective range of the IR Sensor, allowing it to sense objects that are much further away.

PARAMETER	SYMBOL	MIN.	TYP.	MAX.	UNIT	TEST CONDITION	BIN NO.
Aperture Radiant Incidence	Ee	0.088		0.168	mW/cm²	I_F = 20mA	BIN B
		0.112		0.204			BIN C
		0.136		0.240			BIN D
		0.160		0.288			BIN E
		0.192					BIN F
Radiant Intensity	I_E	0.662		1.263	mW/sr	I_F = 20mA	BIN B
		0.842		1.534			BIN C
		1.023		1.805			BIN D
		1.203		2.165			BIN E
		1.444					BIN F
Peak Emission Wavelength	λ_{Peak}		940		nm	I_F = 20mA	
Spectral Line Half-Width	$\Delta\lambda$		50		nm	I_F = 20mA	
Forward Voltage	V_F		1.2	1.6	V	I_F = 20mA	
Reverse Current	I_R			100	μA	V_R = 5V	
Viewing Angle (See FIG.6)	$2\theta_{1/2}$		40		deg.		

Figure 15: Forward Voltage of the IR Emitter (LTE-302)

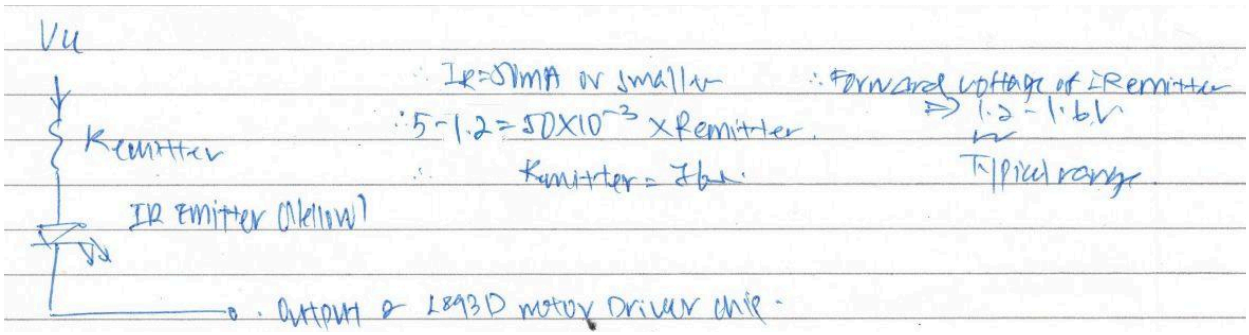


Figure 16: Maximum Biasing Resistor Calculations (IR Emitter)

In our calculations, we used the typical forward voltage for the IR Emitter (1.2V). We also assumed that the L293 Channel Driver is 0V and Vcc is 5V.

The maximum biasing resistor that we can use is 76Ω.

2.6.2 Finalised Resistor Values for IR Emitter

1.0	10	100	1.0 k	10 k	100 k	1.0 M
1.1	11	110	1.1 k	11 k	110 k	1.1 M
1.2	12	120	1.2 k	12 k	120 k	1.2 M
1.3	13	130	1.3 k	13 k	130 k	1.3 M
1.5	15	150	1.5 k	15 k	150 k	1.5 M
1.6	16	160	1.6 k	16 k	160 k	1.6 M
1.8	18	180	1.8 k	18 k	180 k	1.8 M
2.0	20	200	2.0 k	20 k	200 k	2.0 M
2.2	22	220	2.2 k	22 k	220 k	2.2 M
2.4	24	240	2.4 k	24 k	240 k	2.4 M
2.7	27	270	2.7 k	27 k	270 k	2.7 M
3.0	30	300	3.0 k	30 k	300 k	3.0 M
3.3	33	330	3.3 k	33 k	330 k	3.3 M
3.6	36	360	3.6 k	36 k	360 k	3.6 M
3.9	39	390	3.9 k	39 k	390 k	3.9 M
4.3	43	430	4.3 k	43 k	430 k	4.3 M
4.7	47	470	4.7 k	47 k	470 k	4.7 M
5.1	51	510	5.1 k	51 k	510 k	5.1 M
5.6	56	560	5.6 k	56 k	560 k	5.6 M
6.2	62	620	6.2 k	62 k	620 k	6.2 M
6.8	68	680	6.8 k	68 k	680 k	6.8 M
7.5	75	750	7.5 k	75 k	750 k	7.5 M
8.2	82	820	8.2 k	82 k	820 k	8.2 M
9.1	91	910	9.1 k	91 k	910 k	9.1 M

Figure 17: Standard Resistor Table (±5%) (The Open University, n.d.)

Table G: Theoretical vs Implemented Resistor Values

	Minimum	**Actual**
IR Emitter	76Ω	150Ω || 220Ω ≈ 89.19Ω (2d.p)

Based on the standard resistor table in Figure 17, the closest resistor is 82Ω. However, it is not provided in our lab. As a result, we connected 150Ω and 220Ω in a parallel configuration to give us an overall resistance of 89.19Ω, which is quite close to the minimum value that we calculated.

2.6.3 Finalised Resistor Values for IR Detector

For the IR Detector voltage divider circuit, a 10kΩ resistor was utilized instead of the 8.2kΩ resistor that we used in the studio sessions. We observed that using a 10kΩ resistor helped to improve the stability of the readings from the IR detector as compared to the 8.2kΩ resistor.

2.6.4 Other Considerations

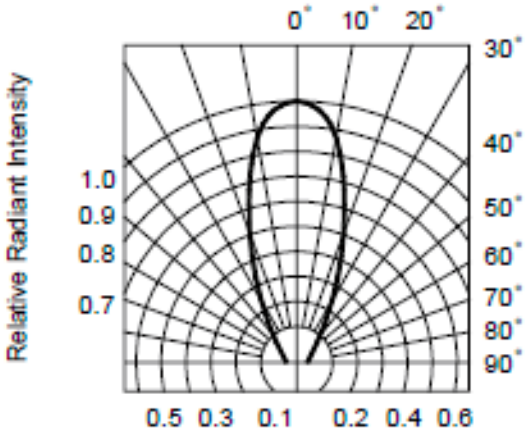


FIG.6 RADIATION DIAGRAM

Figure 18: IR Emitter (LTE-302) Radiation Diagram

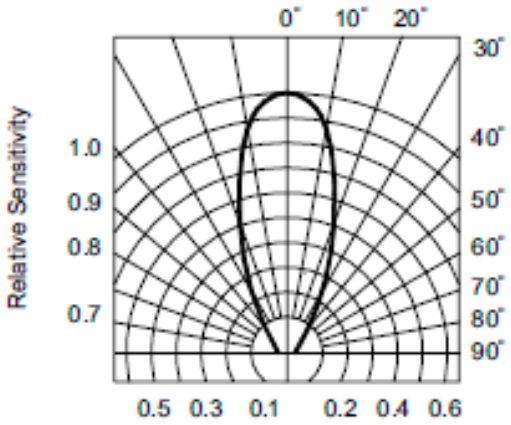


FIG.5 SENSITIVITY DIAGRAM

Figure 19: IR Detector (LTE-301) Sensitivity Diagram

As shown in Section 1.2, the IR Emitter (LTE-302) and IR Detector (LTR-301) should be placed one column apart on the breadboard as there might be indirect IR leakage if the IR Detector is within the radiation cone (±30°) of the Emitter.

Conversely, if the IR Emitter and Detector are positioned too far apart, the accuracy and reliability of the IR Sensor will be affected as both components are not operating in their optimal range as shown in Figure 18 and Figure 19.

3. Algorithm Design

3.1 Flowchart of Algorithm

Figure 20 shows the overall flowchart of our mBot's algorithm. The algorithm continuously reads data from the mBot's line sensor as well as the ultrasonic and IR sensors to decide between two primary states: Navigation Mode and Challenge Mode.

In Navigation Mode, the robot continuously moves straight through the maze while detecting the maze walls. The ultrasonic sensor on the left, measures the distance to the left wall, while the IR sensor on the right, measures the distance to the right wall. Based on these readings, if the mBot is too close to the left wall, it will perform a right nudge to adjust its position. If the mBot is too close to the right wall, it will perform a left nudge. Otherwise, it will continue moving forward. These checks ensure that the mBot remains centered within the maze.

Once the line sensor detects a black line, it will switch to the Challenge Mode. In Challenge Mode, the colour detector sequentially activates the RGB LEDs and the LDR reads the reflected light intensity. These readings are normalized and with these values, the algorithm will determine the detected colour. It will then perform the corresponding manoeuvre defined in Figure 20 based on the identified colour.

After performing the corresponding manoeuvre, the mBot switches back to Navigation Mode, and continues moving forward while detecting the black line. This cycle repeats until a white colour waypoint is detected, which indicates the end of the maze and terminates the run. The mBot will stop and play celebratory music.

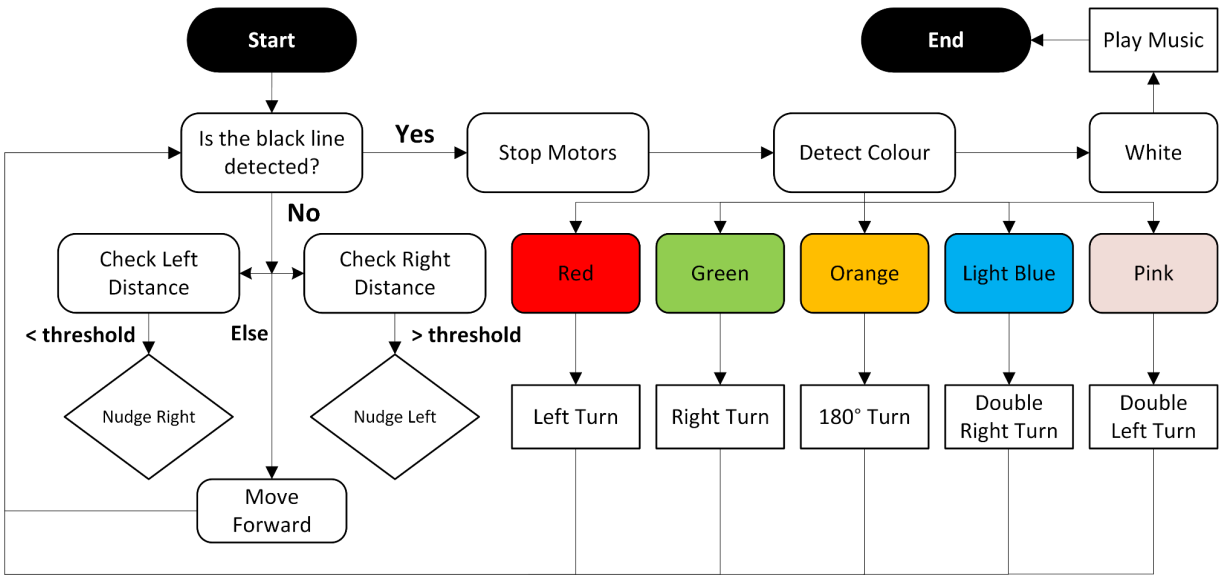


Figure 20: Flowchart of Overall Algorithm

4. Subsystem Algorithm

4.1 Navigation

4.1.1 Wall Avoidance

To allow the mBot to move through the maze while maintaining a centred position between the walls using feedback from the ultrasonic and IR sensors. We created several functions - moveStraight(), nudgeLeft(), nudgeRight() and moveForward(). The navigation logic is mainly handled by the moveStraight() function. When no black line is detected, the mBot continuously calls this function to maintain straight movement.

In each loop, the mBot compares distance readings from the left (ultrasonic sensor) and right (IR sensor) to determine whether correction is needed.

- If no left echo is detected or IR reading is above threshold, it will not make any false corrections and just move forward. This prevents the mBot from over-correcting when either sensor returns an invalid value, such as no wall conditions.
- If the left wall is too close (distance_from_left < threshold), the mBot will call nudgeRight() to shift slightly away from the wall. In nudgeRight(), the left motor runs at full speed (motorSpeed = -255), while the right motor runs slower (nudgeSpeed = 180).
- If the right wall is too close (distance_from_right > threshold), the mBot will call nudgeLeft() to shift slightly away from the wall. In nudgeLeft(), the left motor runs slower (nudgeSpeed = -180), while the right motor runs at full speed (motorSpeed = 255).
- If neither conditions are met (mBot is centred within acceptable range), the mBot will call moveForward() to allow both motors to run at full speed (|motorSpeed| = 255).

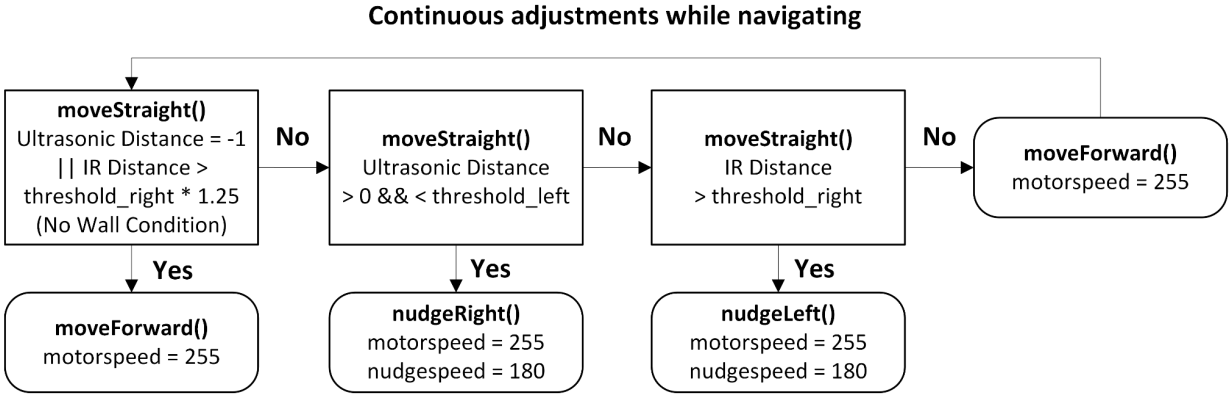


Figure 21: Flowchart of Wall Avoidance Algorithm

```
void moveStraight(){
     double distance_from_left = getUltrasonic(); //Closer corresponds to
smaller distance_from_left
     double distance_from_right = getIR(); //Closer corresponds to larger
distance_from_right

    //Condition to account for no walls on either side
    //If either wall too far, do not make any nudges and move forward
    if (distance_from_left== -1 || distance_from_right>threshold_right *
1.25) {
      moveForward();
    }
    //If too close to left wall, nudge right
   else if (distance_from_left>0 && distance_from_left<threshold_left) {
       nudgeRight();
       delay(10);
    }
    //If too close to right wall, nudge left
    else if (distance_from_right>threshold_right) {
       nudgeLeft();
       delay(10);
    }
    //If in centre, do not nudge and move forward
    else{
      moveForward();
    }
```

Code 1: Overall Wall Avoidance Algorithm

```
//Function to swerve left, left wheel moves slower than right wheel
void nudgeLeft() {
 leftMotor.run(-nudgeSpeed);
 rightMotor.run(motorSpeed); //Will swerve left
}
//Function to swerve right, right wheel moves slower than left wheel
void nudgeRight() {
  leftMotor.run(-motorSpeed);
  rightMotor.run(nudgeSpeed); //Will swerve right
}
```

Code 2: Nudging Algorithm

```
void stopMotor() { //Function to stop both motors
  leftMotor.stop();
  rightMotor.stop();
}
void moveForward() { //Function to move forward
  leftMotor.run(-motorSpeed);
  rightMotor.run(motorSpeed);
}
```

Code 3: Moving and Stopping Code

4.2 Proximity Sensors

4.2.1 Ultrasonic Sensor

To get the left distance for our Navigation algorithm, it calls getUltrasonic(), where the ultrasonic sensor would measure the distance to the left wall. It works by emitting a short ultrasonic pulse and measuring the time taken for the echo to return, the distance would then be computed using the formula shown in the code. The timeout limit is set so that the function returns immediately when no echo is detected. The resulting distance would then be used in the Navigation algorithm to ensure that the mBot does not collide with the left wall.

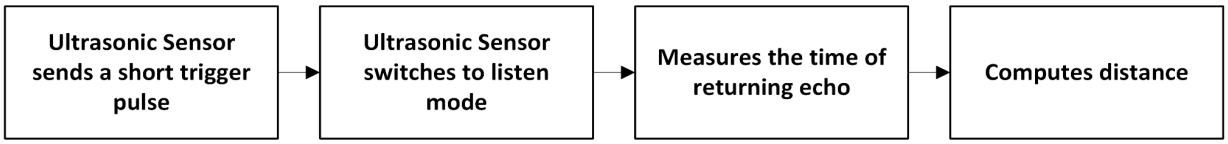


Figure 22: Ultrasonic Sensor Function

```
#define TIMEOUT 1500         //Max microseconds to wait
#define SPEED_OF_SOUND 340   //Update according to experiment
#define ULTRASONIC 10        //D10 = Ultrasonic
double getUltrasonic() {
  pinMode(ULTRASONIC, OUTPUT); //Configure ultrasonic pin
  digitalWrite(ULTRASONIC, LOW); //Send ping
  delayMicroseconds(2);
  digitalWrite(ULTRASONIC, HIGH);
  delayMicroseconds(2);
  digitalWrite(ULTRASONIC, LOW);
  pinMode(ULTRASONIC, INPUT); //Configure ultrasonic pin

  long duration = pulseIn(ULTRASONIC, HIGH, TIMEOUT);
  double distance_from_left = 0;
  if (duration > 0) { // If echo is detected
     distance_from_left = duration / 2.0 / 1000000 * SPEED_OF_SOUND *
100; //Time taken to travel to left wall is half of total distance
  }
  else { //If no echo is detected within timeout duration (no walls)
    return -1; }
  return distance_from_left;
}
```

Code 4: Ultrasonic Sensor Code

4.2.2 IR Sensor

In addition to the ultrasonic sensor, the use of the IR Sensor (emitter and detector) allows the mBot to navigate through the maze without colliding into the right wall. To get the right distance for our Navigation algorithm, it calls getIR(), where the IR sensor would measure the distance to the right wall. It works by emitting infrared light and measuring the reflected amount. However, the detector would give inconsistent readings due to the effect of ambient lighting, hence the difference between the ambient reading and measured reading was used instead. This process is described in Figure 23. The calculated value would then be used in the Navigation algorithm in conjunction with the ultrasonic sensor to ensure the mBot is centred at all times.

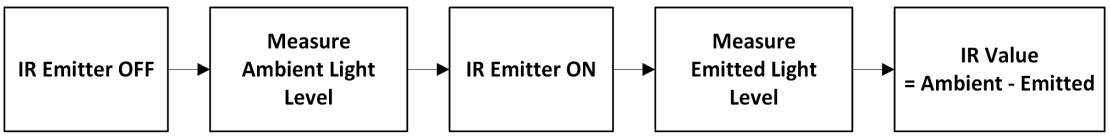


Figure 23: IR Sensor Function

```
#define IR_DETECTOR 1  //A1
#define IR_WAIT 1
void shineIR() { //Function for turning on the IR Emitter only
  digitalWrite(DECODER_SELECT_1, LOW);
  digitalWrite(DECODER_SELECT_2, LOW);
}
void offIR() { //Function for turning off the IR Emitter only
  digitalWrite(DECODER_SELECT_1, HIGH);
  digitalWrite(DECODER_SELECT_2, HIGH);
}
double getIR() {
  offIR(); //Switches off IR Emitter
  delay(IR_WAIT);
  double ambientIR = analogRead(IR_DETECTOR); //Measures ambient level
  shineIR(); //Switches on IR Emitter
  delay(IR_WAIT);
  double reading = analogRead(IR_DETECTOR); //Measures emitted level
  double IR_val = ambientIR-reading; //Computes difference

  return IR_val;
}
```

Code 5: IR Sensor Algorithm

4.3 Line Detection

The mBot's line sensor consists of two infrared sensors (S1 and S2), mounted at the front and under the mBot. These sensors function similarly to the IR sensor described previously, but this pair of sensors detect whether the surface below is black or white based on the reflected IR light. In the main loop of our code, the line sensor's state is continuously being monitored as it is used as a condition to perform certain actions. S1_OUT_S2_OUT refers to both infrared sensors detecting white, hence the loop calls for moveStraight(). !S1_OUT_S2_OUT refers to either or both infrared sensors detecting black, hence the mBot stops and the loop switches to Challenge Mode where it will perform the corresponding manoeuvre according to the detected colour.

```
int status = 0;  //Global status; 0 = do nothing, 1 = mBot runs
void loop() {
  sensorState = lineFinder.readSensors();
  if (status == 0) { //mBot has not reached end of maze, continue moving straight and checking for challenges
      if (sensorState != S1_OUT_S2_OUT) { //Detects black line, enter CHALLENGE mode
      stopMotor();
      action(detectColor()); //mBot goes into CHALLENGE mode
    } else {
      moveStraight(); //Continue moving straight if no black line
    }
  }
  else { //End of maze condition
      stopMotor(); //Stops moving and stops checking for challenges
  }
}
```

Code 6: Line Detection Algorithm

4.4 Colour Sensing

```
PaperColours detectColor()
{
  //turn on one colour at a time and LDR reads 5 times
  for (int c = 0; c <= 2; c++) {

    if (c == 0) shineRed(); //Turns on the red LED
    else if (c == 1) shineGreen(); //Turns on the green LED
    else shineBlue(); //Turns on the blue LED

    delay(RGBWait);

    colourArray[c] = getAvgReading(5);
    //get the average of 5 consecutive readings
    //for the current colour and return an average

    colourArray[c] = (colourArray[c]-blackArray[c])/(greyDiff[c])*255;
    //Normalize the values and store it within the colourArray
    //the average reading returned minus the lowest value divided
    //by the maximum possible range, multiplied by 255 will give
    //a value between 0-255, representing the value for the current
    //reflectivity (i.e. the colour LDR is exposed to)

    offLED();
    delay(RGBWait);
  }
  return getColour();
}
```

Code 7: Overall Colour Sensing Algorithm

4.4.1 detectColor()

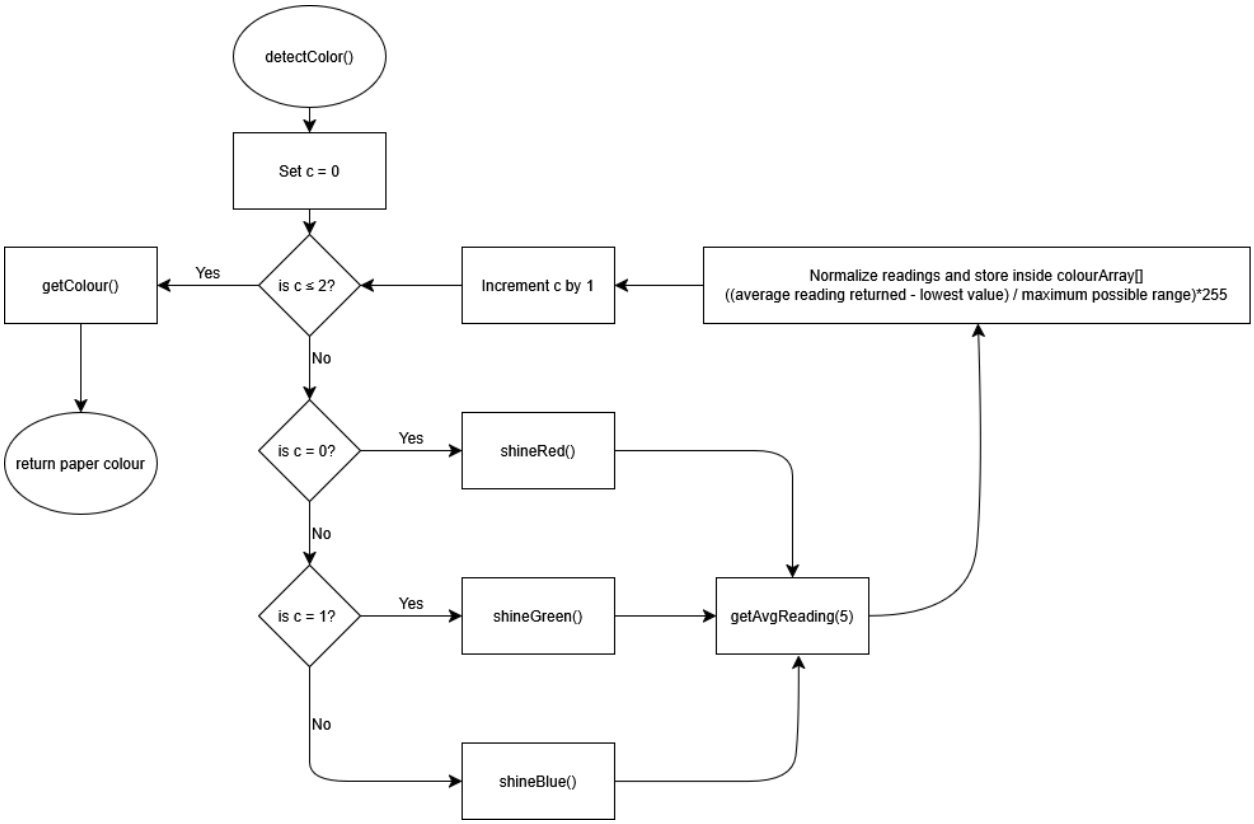


Figure 24: detectColor() Function

Colour	Interpretation
Red	Left-turn
Green	Right turn
Orange	180° turn within the same grid
Pink	Two successive left-turns in two grids
Light Blue	Two successive right-turns in two grids

Figure 25: Corresponding manoeuvres based on detected colour

When the line sensor detects a black line, it will switch to the Challenge Mode. In this Mode, the mBot will be required to detect the colour of the paper and perform a specific manoeuvre as shown in Figure 25.

As shown in Figure 24, the colour sensing algorithm detectColor() operates in 2 phases, namely

1) RGB Detection - getAvgReading()
2) Colour Classification - getColour()

The detectColor() function loops through the 3 LEDs and activates the LEDs based on the counter (c) in the loop. The LEDs are activated using the decoder as shown in Section 2.5.

1) If c = 0, the red LED is activated using shineRed()

```
void shineRed() { //Function to turn on Red LED only
  digitalWrite(DECODER_SELECT_1, HIGH);
  digitalWrite(DECODER_SELECT_2, LOW);
}
```

Code 8: shineRed()

2) If c = 1, the green LED is activated using shineGreen()

```
void shineGreen() { //Function to turn on Green LED only
  digitalWrite(DECODER_SELECT_1, HIGH);
  digitalWrite(DECODER_SELECT_2, HIGH);
}
```

Code 9: shineGreen()

3) If c = 2, the blue LED is activated using shineBlue()

```
void shineBlue() { //Function to turn on Blue LED only
  digitalWrite(DECODER_SELECT_1, LOW);
  digitalWrite(DECODER_SELECT_2, HIGH);
}
```

Code 10: shineBlue()

The LEDs will be turned off using the offLED() function, which sets the select pins of the decoder such that the data output pins that are connected to the LEDs as shown in Section 2.5 are pulled to HIGH.

```
void offLED() { //Function to turn off all LEDs
  digitalWrite(DECODER_SELECT_1, LOW);
  digitalWrite(DECODER_SELECT_2, LOW);
}
```

Code 11: offLED()

4.4.2 getAvgReading()

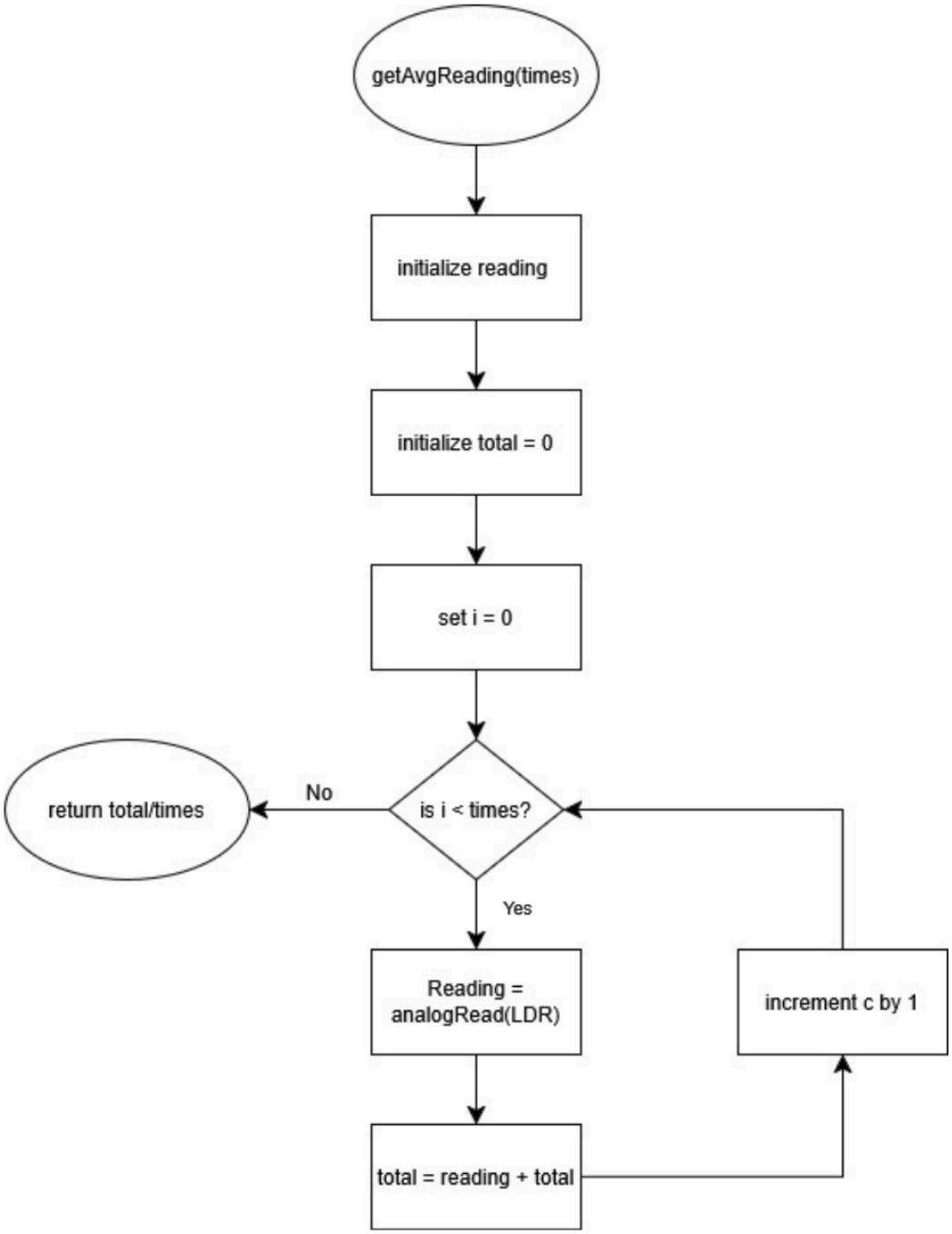


Figure 26: getAvgReading() Function

When the LED is activated, the mBot will call the function getAvgReading(), which gets the average LDR readings using analogRead() as shown in Section 2.5. analogRead() will then map the analog voltage range from 0V to 5V, to 0 to 1023. It will then return the average LDR reading using the formula (total LDR reading/number of times read). It will then return the reading back to the detectColor() function.

4.4.3 getColour()

Once the average LDR reading is returned back into the detectColor() function, the values are assigned to colourArray[], which stores the red, blue and green readings of the detected coloured paper. Thereafter, the detectColor() function will normalize the readings based on the calibrated greyDiff[] array as shown in Section 5.1.

$$\frac{average\ reading\ returned\ (colourArray[c]) - lowest\ value\ (blackArray[c])}{maximum\ possible\ range\ (greyDiff[c])} * 255$$

Equation 1: Normalization Formula

Using equation 1, the LDR analogRead() value of 0 to 1023 will be scaled to 0 to 255 based on our initial white and black colour calibration as shown in Section 5.1, this mapping would allow the getColour() function to distinguish the coloured papers using the RGB format.

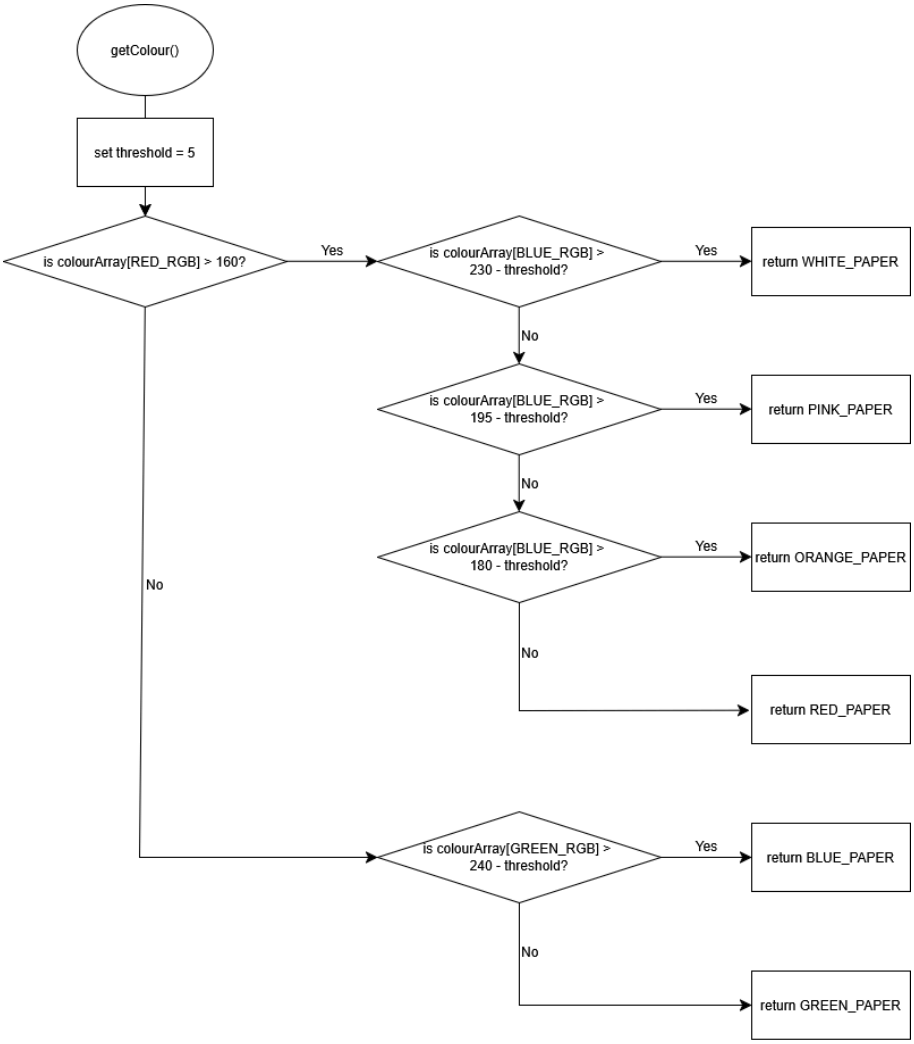


Figure 27: getColor() Function

```
//LED readings
#define RED_RGB 0
#define BLUE_RGB 1
#define GREEN_RGB 2
```

Code 12: Definition for the 3 different RGB colours

```
typedef enum {
  WHITE_PAPER = 1,
  BLACK_PAPER = 2,
  RED_PAPER = 3,
  GREEN_PAPER = 4,
  BLUE_PAPER = 5,
  PINK_PAPER = 6,
  ORANGE_PAPER = 7,
} PaperColours;
```

Code 13: Definition for coloured paper

```
PaperColours getColour() { //Function to determine colour detected
  int threshold = 5;
  if (colourArray[RED_RGB] > 160) { //Values obtained from calibration
    if (colourArray[BLUE_RGB] > (230 - threshold)) {
      return WHITE_PAPER;
    } else if (colourArray[BLUE_RGB] > (195 - threshold)) {
      return PINK_PAPER;
    } else if (colourArray[BLUE_RGB] > (180 - threshold)) {
      return ORANGE_PAPER;
    } else {
      return RED_PAPER;
    }
  } else {
    if (colourArray[GREEN_RGB] > (240 - threshold)) {
      return BLUE_PAPER;
    } else {
      return GREEN_PAPER;
    }
  }
}
```

Code 14: getColour() Algorithm

Figure 27 shows how the overall getColour() function works. The getColour() function will first check if the colourArray[RED_RGB] value is bigger than 160.

If it is bigger than 160, it will return an enumerated value to the detectColor() function based on the colourArray[BLUE_RGB] value through a series of nested conditions. We also set a threshold value of 5 to account for the fluctuations of the LDR sensor for different environmental conditions.

Table H: getColour()'s if-else conditions for colourArray[BLUE_RGB]

Conditions	Return Value
colourArray[BLUE_RGB] > (230 - threshold)	WHITE_PAPER [1]
colourArray[BLUE_RGB] > (195 - threshold)	PINK_PAPER [6]
colourArray[BLUE_RGB] > (180 - threshold)	ORANGE_PAPER [7]
colourArray[BLUE_RGB] ≤ (180 - threshold)	RED_PAPER [3]

If the colourArray[RED_RGB] value is smaller or equal than 160, it will return an enumerated value to the detectColor() function based on the colourArray[GREEN_RGB] value through a series of nested conditions. Similarly, a threshold value of 5 is used to account for the fluctuations of the LDR sensor for different environmental conditions.

Table I: getColour()'s if-else conditions for colourArray[GREEN_RGB]

Conditions	Return Value
colourArray[GREEN_RGB] > (240 - threshold)	BLUE_PAPER [5]
colourArray[GREEN_RGB] ≤ (240 - threshold)	GREEN_PAPER [4]

The rationale of the threshold values for each coloured paper will be further discussed in Section 5.1.

4.5 Action Execution

4.5.1 General Action

After the black line has been detected by the line sensor, the mBot would perform corresponding manoeuvres according to the detected colour. The main overall algorithm is done using the action() function, which calls on other functions to perform the manoeuvres. The action() function takes in a parameter, which is passed by the detectColor() function discussed in Section 4.4.1. Each action called is a specific movement sequence which makes use of the turning, compound movement and celebratory functions discussed below.

```
void action(PaperColours c) { //Function for corresponding manoeuvres
  if (c == RED_PAPER) {
    turnLeft();
  } else if (c == BLUE_PAPER) {
    doubleRightTurn();
  } else if (c == GREEN_PAPER) {
    turnRight();
  } else if (c == PINK_PAPER) {
    doubleLeftTurn();
  } else if (c == ORANGE_PAPER) {
    uTurn();
  } else if (c == WHITE_PAPER) {
    celebrate();
  } else {
    return;
  }
}
```

Code 15: Overall Action Algorithm

4.5.2 Turning

We created a general turning function, which can be applied across other functions such as turnLeft(), turnRight() and uTurn(). Using the turn() function, the mBot is able to perform on-the-spot rotations by running both motors in the same direction according to a time duration.

```
#define turning_time_ms 360 //The time duration (ms) for turning 90°
void turn(int dir, double deg) { //0 for left, 1 for right
```

```
  if (dir == 0) {
    leftMotor.run(motorSpeed); //+ve: wheel turns clockwise
    rightMotor.run(motorSpeed); //+ve: wheel turns clockwise
    delay((turning_time_ms / 90) * deg); //Keep turning left for this
time duration
    stopMotor();
  }
  if (dir == 1) {
    leftMotor.run(-motorSpeed); //-ve: wheel turns clockwise
    rightMotor.run(-motorSpeed); //-ve: wheel turns clockwise
    delay((turning_time_ms / 90) * deg); //Keep turning left for this
time duration
    stopMotor();
  }
}
```

Code 16: General Turn Algorithm

4.5.3 Compound Movement

In addition to the generic turn function, we created two compound movement functions, which are doubleLeftTurn() and doubleRightTurn(). This allows the mBot to turn, move forward and turn again without hitting the center pole.

```
#define double_time_forward 790 // Time taken to move a certain distance
void doubleLeftTurn() { // Code for double left turn
  turnLeft();
  moveStraight();
  delay(double_time_forward); // Time taken to move one block
  turnLeft();
}

void doubleRightTurn() { // Code for double right turn
  turnRight();
  moveStraight();
  delay(double_time_forward);
  turnRight();
}
```

Code 17: Compound Movement Algorithm

4.5.4 Celebratory Action

Once the mBot detects a white coloured paper at the end of the maze, the celebrate() function will be called, which will play a simple melody.

```
void celebrate() {
  int tempo = 180;
  int gap = 50;

  buzzer.tone(392, tempo); delay(tempo + gap);
}
```

Code 18: Celebratory Music Code

5. Sensor Calibration

5.1 Colour Sensor

The calibration process of the colour sensor is achieved using the setBalance() function, which sets reference values for white and black paper samples.

The calibration process involves shining red, green and blue light onto both the white and black paper samples. For each colour, the LDR records multiple readings, and the average of 30 samples would be stored in whiteArray[].

The same process is repeated for the black paper sample, but stored into blackArray[]. The difference between the white and black readings across the three channels are computed and stored in greyDiff. This represents the range of detectable reflectivity values for each channel, and it will be used to normalize the colour paper readings in Section 4.4.3.

```
void setBalance() { //Function for colour calibration
  Serial.println("Put White Sample For Calibration ...");
  delay(5000);  //Set white balance

  for (int i = 0; i <= 2; i++) { //Shines red, green and blue to set the
maximum reading to whitearray
    if (i == 0) shineRed();
    else if (i == 1) shineGreen();
    else shineBlue();
    delay(RGBWait);
    whiteArray[i] = getAvgReading(30); //Returns the average of 30 scans
    offLED();
    delay(RGBWait);
  }
  //Sets black balance
  Serial.println("Put Black Sample For Calibration ...");
  delay(5000);  //Shines red, green and blue to the black array
  for (int i = 0; i <= 2; i++) {
    if (i == 0) shineRed();
    else if (i == 1) shineGreen();
    else shineBlue();
    delay(RGBWait);
    blackArray[i] = getAvgReading(30); //Get average reading
    offLED();
```

```
    delay(RGBWait);

    //The difference between the maximum and the minimum gives the range
    greyDiff[i] = whiteArray[i] - blackArray[i];
  }
```

Code 19: Calibrating the Colour Sensor using setBalance()

```
int getAvgReading(int times) { //Function to get average LDR readings
  int reading;
  int total = 0;
  //Averages the requested number of scans required
  for (int i = 0; i < times; i++) {
    reading = analogRead(LDR);
    total = reading + total;
    delay(LDRWait);
  }
  //calculates the average and returns it
  return total / times;
}
```

Code 2-: Get average readings for Colour Sensor

5.1.1 Calibration for each coloured paper

```
PaperColours detectColor() {
  for (int c = 0; c <= 2; c++) {
    Serial.print(colourStr[c]);
    Serial.println(int(colourArray[c])); //Shows the value for the
current LED colour, which matches either the R, G or B of the RGB code
}
```

Code 21: Code Snippet of detectColor() to obtain normalized RGB values

After completing the initial white and black calibration, we also needed to determine the optimal threshold values in order for the mBot to distinguish between the 5 main colours as shown in Figure 25. In addition, we also collected the normalized RGB values for white as it is the last colour that the mBot needs to detect before it plays the celebratory tune as shown in Section 4.5.4.

This process required us to collect the average normalized RGB values (10 samples) for each colour using the Serial.print() function as shown in Code 21. These are the results for the different sessions:

Table J: Normalized RGB values for Session 1

Colour	R	B	G
Orange	255	203	152
Red	251	146	139
Blue	187	225	241
White	258	258	260
Green	188	231	216.4
Pink	259	239	239

Table K: Normalized RGB values for Session 2

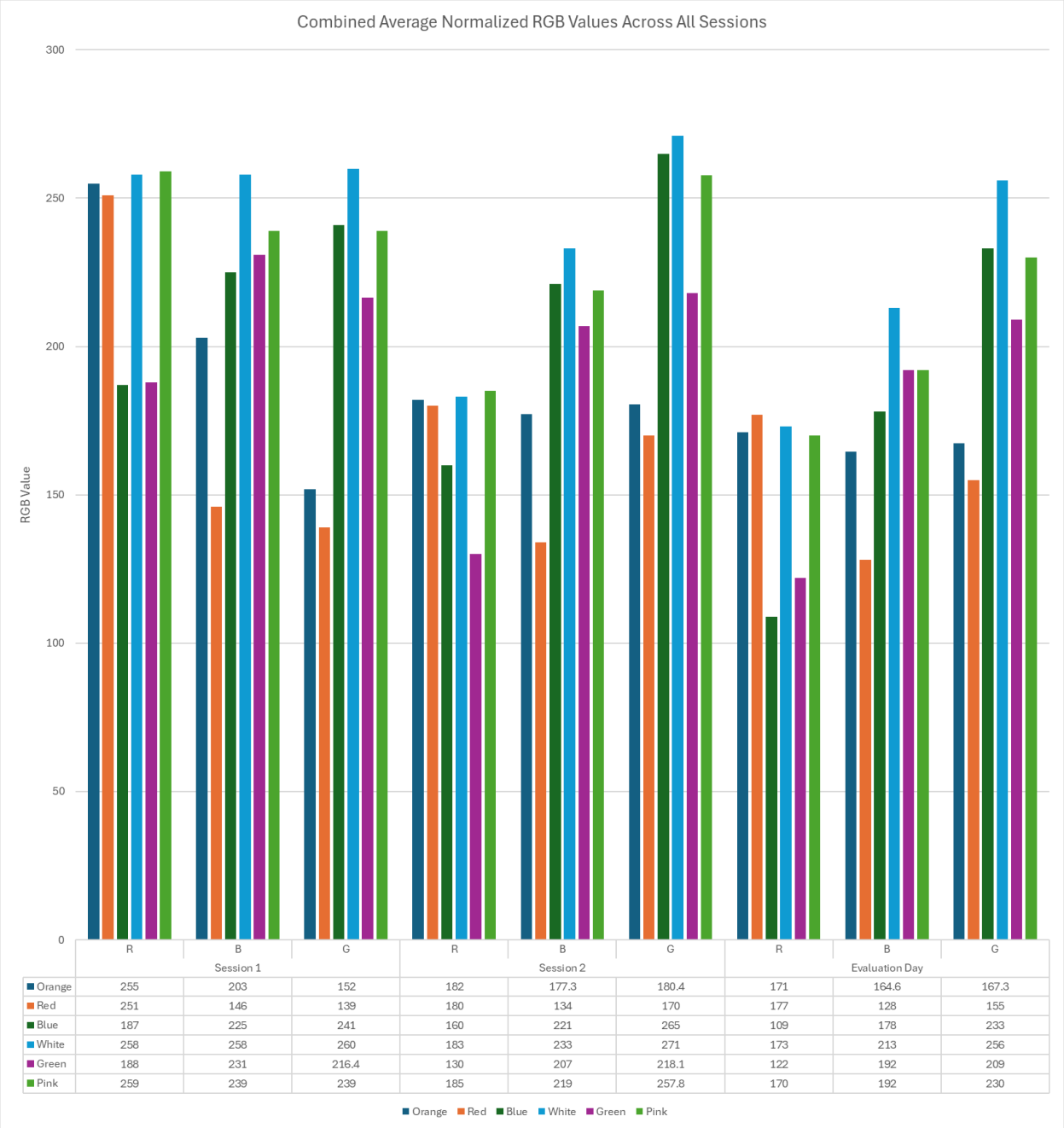
Colour	R	B	G
Orange	182	177.3	180.4
Red	180	134	170
Blue	160	221	265
White	183	233	271
Green	130	207	218.1
Pink	185	219	257.8

Table L: Normalized RGB values for Evaluation day

Colour	R	B	G
Orange	171	164.6	167.3
Red	177	128	155
Blue	109	178	233
White	173	213	256
Green	122	192	209
Pink	170	192	230

*Note: Each RGB value represents the average of 10 measurements. The complete raw calibrations values are shown in Appendix.

5.1.2 Calibration findings



	Session 1			Session 2			Evaluation Day		
	R	B	G	R	B	G	R	B	G
Orange	255	203	152	182	177.3	180.4	171	164.6	167.3
Red	251	146	139	180	134	170	177	128	155
Blue	187	225	241	160	221	265	109	178	233
White	258	258	260	183	233	271	173	213	256
Green	188	231	216.4	130	207	218.1	122	192	209
Pink	259	239	239	185	219	257.8	170	192	230

Figure 28: Normalized RGB values for all 3 sessions

Figure 28 shows the normalized RGB values across all 3 calibration sessions. Even though there are some variances within the data, the differences between the RGB values for each of the coloured paper remain consistent.

Figure 28 also shows that green and blue papers consistently show low red channel values across all 3 sessions, while the other colours (white, pink, orange and red) have much higher red

channel values. By using the red channel values, we are able to distinguish the green and blue papers from the other coloured papers.

To distinguish between the green and blue papers, we can utilize their green channel values. From the data, we can infer that the blue paper consistently produces a higher green channel value as compared to the green paper. As a result, we can utilize the middle value between these 2 values as the threshold value to separate these 2 colours in the algorithm's nested loop as shown in Section 4.4.3.

Conversely, for the other coloured papers, we can employ a similar approach to obtain the threshold values by using the blue channel values.

5.1.2.1 Sources of error

The variances can be attributed to changes in the environmental conditions such as the surrounding light intensity which might affect the readings of the LDR, thereby affecting the collected RGB values.

From our data that we obtained from the calibration, some of the RGB values exceeded 255. This can be attributed to inaccurate calibration of the whiteArray and blackArray in setBalance() as shown in Section 5.1. As a result, in the getColour() code as shown in Section 4.4.3, we implemented a threshold value to compensate for this error.

5.2 IR Sensor

Figure 29 shows the "Vout vs Distance" graph for the IR Sensor. It is observed that Vout increases sharply from 1 cm to 5 cm, and begins to plateau at a voltage of ~3V. This shows that the IR sensor is only accurate up to a distance of 5 cm, and its reliable working distance between the sensor and the maze wall is shown in Figure 30.

Figure 30 shows the region where the IR Sensor is more sensitive, implying that the IR Sensor only provides reliable feedback when the mBot is very close to the wall. Due to this limited sensitivity range, while in the Navigation Mode, the mBot needs to be extremely close to the right wall before the nudgeLeft() function can be triggered to nudge it back to the center.

However, the mBot is at least 15 cm away from either wall when it is placed at the initial starting position. This is outside of the IR Sensor's optimal range, meaning that it cannot provide a precise distance measurement. To compensate for this limitation, a threshold value is first recorded during setup using getIR() which provides the distance to the right wall and an offset value is subtracted to make the sensor more responsive to closer walls.

This calibrated threshold acts as a reference point although the IR Sensor cannot measure the ~15cm accurately, it can still detect relative changes in distance. Thus, if the mBot moves closer to the right wall, the IR Sensor output will increase, allowing it to nudge left.

Because of this limited sensitivity range, the IR Sensor may still fail to detect or respond too late since it requires the mBot to be extremely close to the right wall. This inconsistent behaviour of the IR Sensor means that it must be used in tandem with the ultrasonic sensor, which performs well at longer distances.

```
void setup() {  //Begin serial communication
  delay(100);
  double offset = 2; //Offset to allow mBot some leeway from the centre
  //Setting threshold values that represent being centre of the path
  threshold_right = getIR() - offset;
```

Code 22: Code Snippet of setup() to calibrate IR Sensor

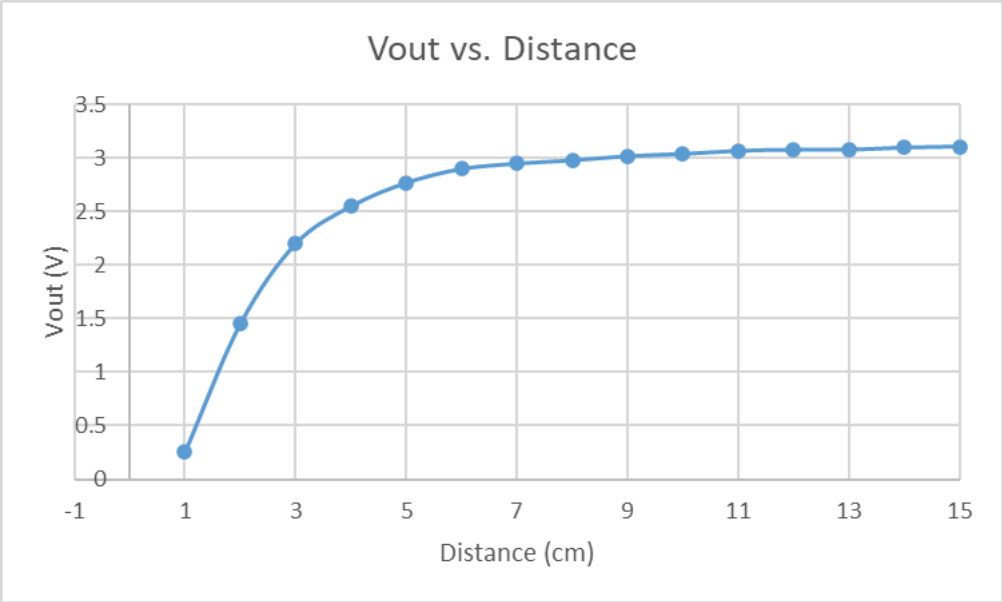


Figure 29: Vout vs Distance graph for IR sensor

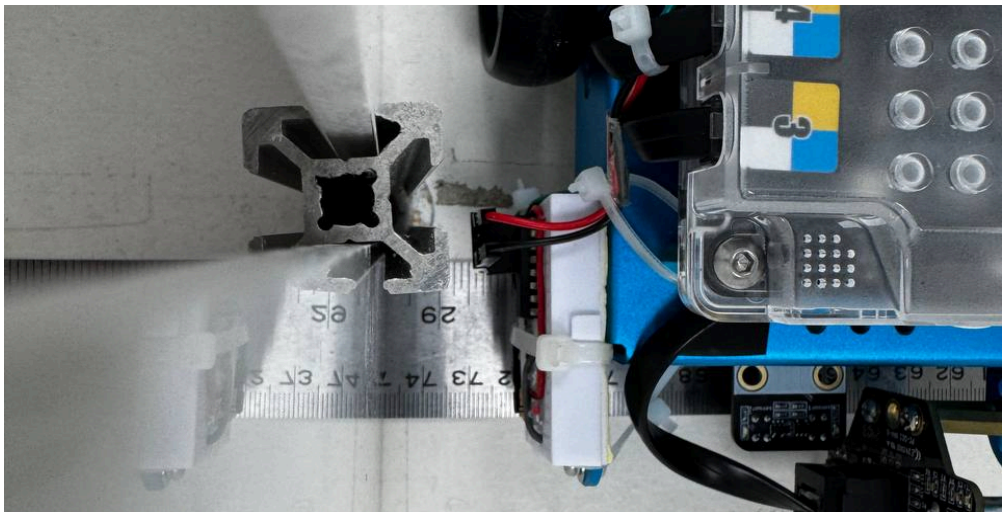


Figure 30: Reliable working distance of IR sensor

6. Challenges Faced

6.1 Inaccuracy of IR Sensor

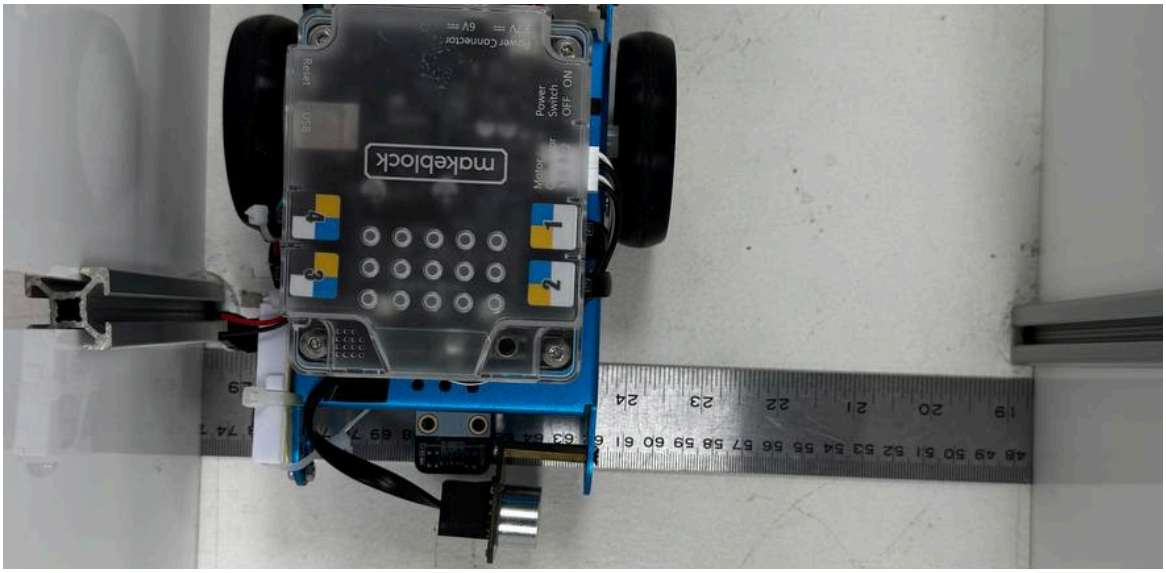


Figure 31: Measuring the distance for our IR Sensor

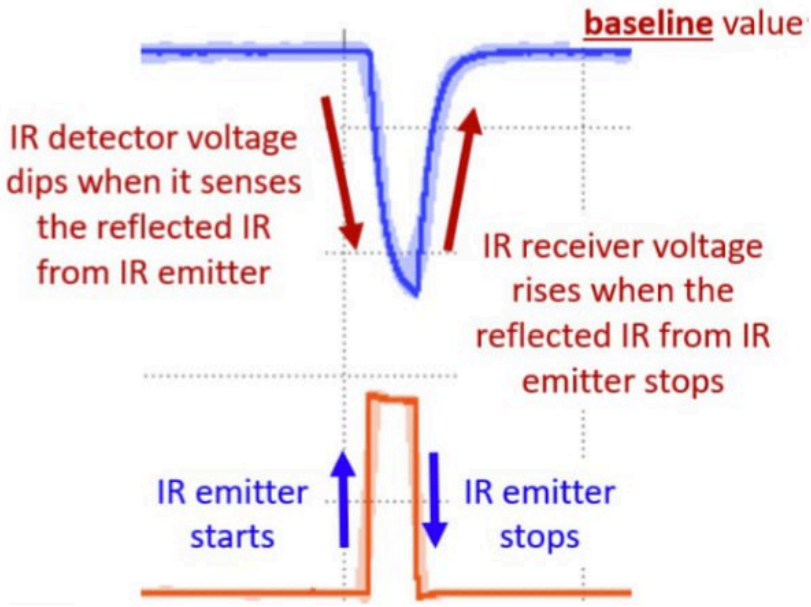


Figure 32: IR Sensor Voltage Variation

Figure 31 shows the distance between the mBot and the right wall before the mBot nudges back into the center of the maze. However, based on our initial testing and experiment, the IR sensor seems to produce unstable readings, likely due to the fluctuations in surrounding ambient IR light. This is illustrated in Figure 32, where the useful signal used is the difference between the baseline value, which is the ambient IR light, and the dipped value, which occurs

when the IR Emitter switches on. This means that the surrounding ambient IR light can cause the baseline value to be adjusted unpredictably, causing the IR reading obtained to be highly sensitive to fluctuations, which results in noisy or unstable measurements.

Initially, we attempted to hardcode a fixed IR distance value that would trigger the mBot to nudge at an appropriate distance. However, this method proved to be unreliable as this value varied under different lighting conditions and could not guarantee consistent performance across all runs.

Therefore, we implemented a dynamic calibration method in our setup() to determine a baseline reference value when the mBot was placed in the center of the maze. This allows our nudging algorithm to work better as it compares against a threshold value which is measured before each run. This method worked well for us as it ensured that the IR Sensor was recalibrated each time, instead of just hardcoding a value.

Despite this improvement, the IR Sensor's limitation in range still posed a challenge. This is evident as the mBot is placed at least 15cm away for the initial starting position, beyond its most accurate range. As such, while it is slightly more stable compared to a hardcoded value, the mBot must still rely on the ultrasonic sensor for more accurate distance measurements.

6.2 Faulty Line Sensor

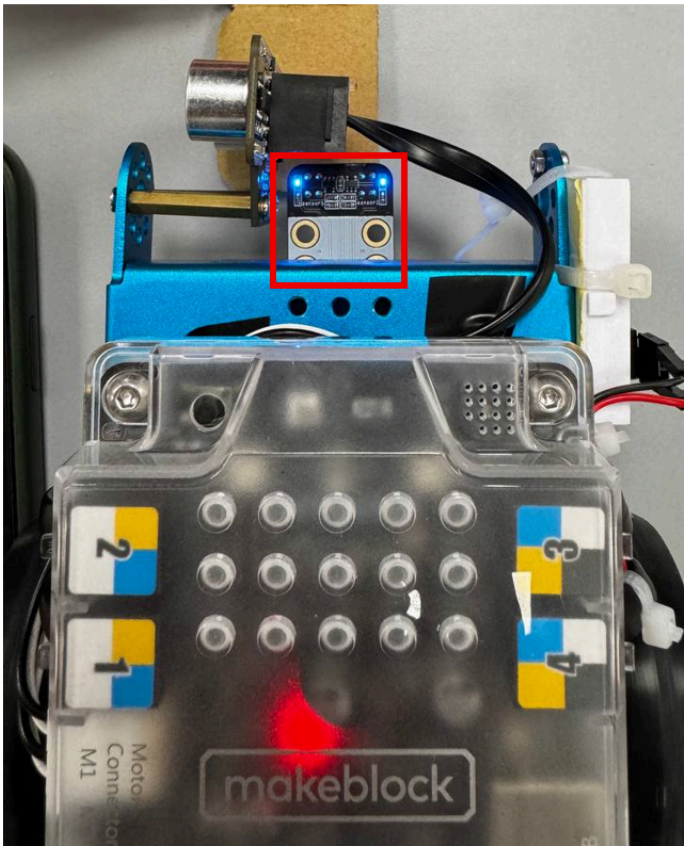


Figure 33: mBot's Line Sensor

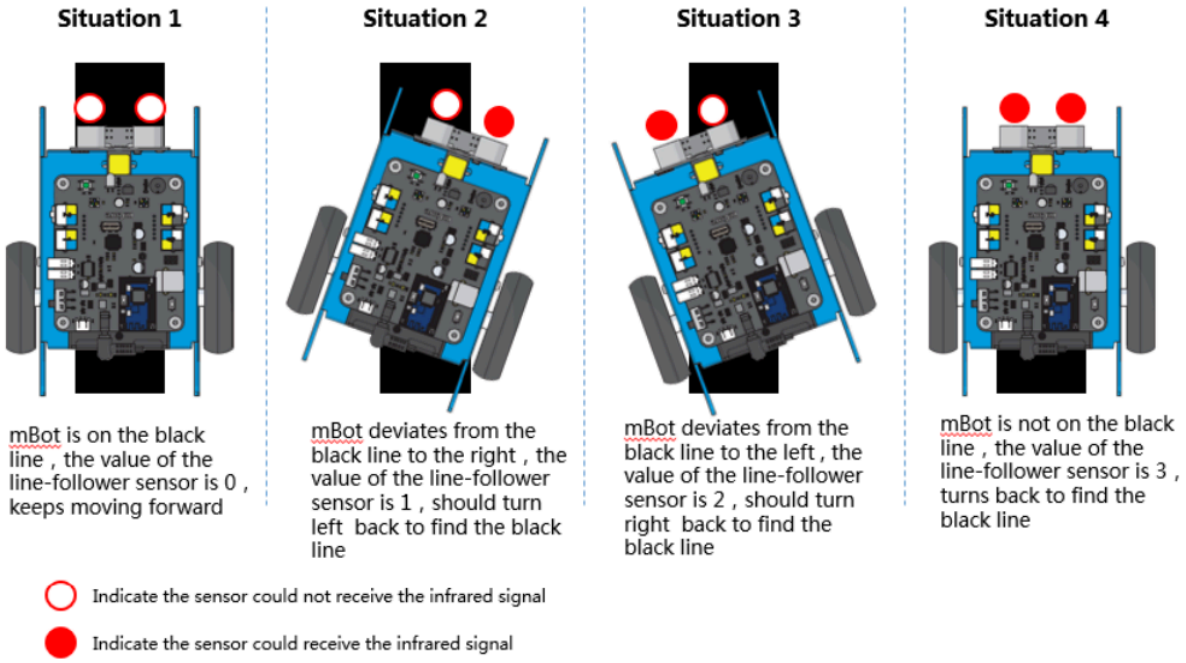


Figure 34: Different states for mBot's Line Sensor (mBlock, n.d.)

Figure 33 shows how the mBot's Line Sensor reacts to either a black or white surface underneath it, with its onboard LEDs lighting up to indicate detection. This visual feedback allows us to verify that the sensor is functioning properly. Figure 34 shows the expected LED response for both black and white surfaces.

During initial testing and experiment on the mBot, we did not realise that the given Line Sensor was malfunctioning as it had intermittent problems, summarised in Table M. We tried to adjust our algorithm and swap the positions of the RJ25 Adaptor cables to remedy the issue. We even tried to readjust the positioning of the line sensor as we had thought that it was not working in its optimal range. We noted down these observations and went to consult the lab technician and requested for a new line sensor to check that we were not mistaken.

Table M: Observations for Line Sensor

Situation	Observation
1	Only 1 LED lights up despite both sensors being placed over black paper. Even when we covered the Line Sensor, neither LED lit up at all
4	While in Navigation mode, the mBot is able to move straight but will overshoot the black line

For situation 1, we replaced the line sensor and conducted multiple tests to ensure that the new component was functioning as expected. Once it was replaced, the mBot functioned as expected and detected both black and white surfaces correctly. This showed the importance of checking and verifying the hardware before algorithmic troubleshooting.

For situation 4, we sought help from our teaching assistant, who suggested that our void loop() was not executing fast enough. This explained why the line detector detected the black line very inconsistently. We observed that when testing with a larger piece of black paper, the mBot stopped at different positions on every test run, indicating that this was a software and not a hardware issue.

We discovered that in the getUltrasound() function, there were multiple delay() calls that we initially implemented to improve the reliability and accuracy of the ultrasonic readings. However, these repeated delay() calls prevented the lineFinder.readSensors() function from consistently detecting the black line, especially when the mBot is moving at higher speeds. As a result, we replaced the delay() calls with delayMicroseconds() in the ultrasonic function. This change significantly increased the scanning speed of the line detector, thereby improving the consistency at which our line detector detected the black line accurately without overshooting.

6.3 Inconsistent readings for Colour Sensor

The Colour Sensor exhibited high inconsistency in readings due to the sensitivity of the LDR. Even minor impacts or vibrations caused by the collision of the mBot and the maze walls would slightly shift the LDR's position, affecting the reflected light intensity and leading to unreliable colour detection. As a result, we had to realign the LDR before each run, which was time-consuming and inefficient.

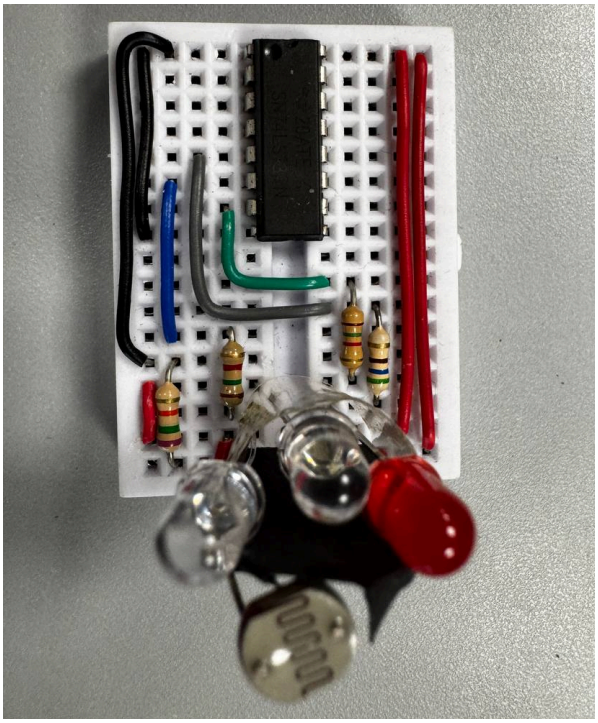
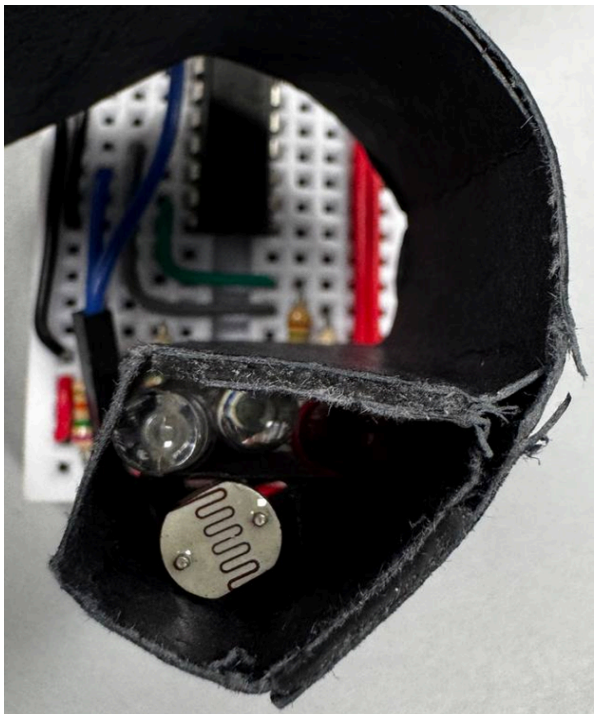
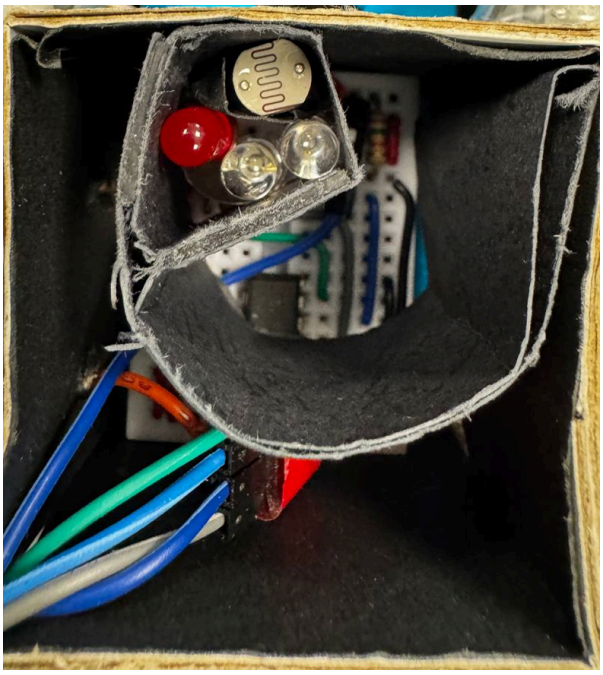
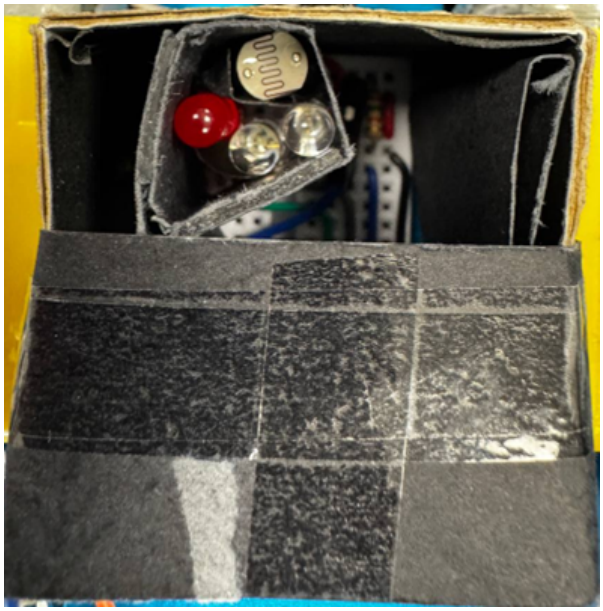
There was an instance where one of the LEDs in the Colour Sensor short-circuited due to collision. This went unnoticed until hardware debugging revealed that the LED was faulty, preventing the Colour Sensor from functioning properly. This highlighted the fragility of our initial hardware setup and the need for improved structural integrity.

To address these issues, we implemented three key improvements to the Colour Sensor:

1. All LEDs were firmly taped down and secured properly to ensure that it would not shift due to collision
2. A black paper divider was placed around the LDR to block stray light from the LEDs, ensuring it only detected reflected light
3. After multiple iterations, we designed a custom casing to firmly hold the LEDs and LDR in place. This design reduced vibrations and significantly improved stability and consistency of the colour readings. It was also a versatile design as it could be reshaped if there was any changes made to the circuit design

These modifications, shown in Table N, improved accuracy and robustness of the Colour Sensor and also reduced the need for frequent recalibration. It was also performing well even without side skirting around the chassis of the mBot, which proved the reliability and innovative design.

Table N: Modifications done to the Colour Sensor

Initial Design	Iteration #1
	
Iteration #2	**Final Design**
	

6.4 Uneven Turning Accuracy

We realised that mBot's turning movement is not consistent at all times due to differences in both motor torques, friction or battery level. These inconsistencies resulted in the mBot overturning or underturning depending on the motor conditions. These cumulative turning errors would accumulate as the mBot navigates further into the maze, causing alignment drift over time.

While testing the mBot, we also realised that certain turns could not be repeated with the same accuracy even though it was performing the same manoeuvre. This prompted us to adjust our algorithm such that we achieve an optimal turning_time value which is used in our turn functions.

For our compound turns, our initial algorithm consisted of nudging while it is performing the turns to ensure that it does not collide with the walls during these complex turns. However, we realised that it was better to achieve a well-calibrated turning angle than to rely on the proximity sensors during these turns as there might be erratic nudging caused by the close proximity between the mBot and the maze walls at those points.

```
void doubleLeftTurn() { //Function to do double left turn
  turnLeft();
  moveForward();  //
  delay(double_time_forward);
  turnLeft();
}
```

Code 23: Code Snippet of doubleLeftTurn()

7. Work Division

Name	Role
Pei Ling	- **IR Sensor Design and Assembly** - **IR Sensor Algorithm** - **Troubleshooting of Overall Code** - **Contributed to the Report**
Gavin	- **Overall Hardware Assembly** - **Colour Sensor Design and Assembly** - **Troubleshooting of Overall Hardware** - **Troubleshooting of Overall Code** - **Contributed to the Report**
Lim Sin	- **Main Algorithm Programmer** - **Navigation, Line Detection, Action Execution, Setup and Looping Algorithm** - **Challenge and Movement Algorithm** - **Troubleshooting of Overall Code** - **Contributed to the Report**
Wei Xiong	- **Main Algorithm Programmer** - **Ultrasonic Sensor, Colour Sensing Algorithm** - **Colour Sensor Design and Assembly** - **Troubleshooting of Overall Code** - **Contributed to the Report**

8. References

The Open University. (n.d.). *2.4 Colour coding and standard resistor values*. Retrieved from The Open University: https://www.open.edu/openlearn/science-maths-technology/an-introduction-electronics/content-section-2.4

mBlock. (n.d.). *Simple line follow program*. https://allemachenmint.de/storage/media/lessons/makeblock/mbot/mbot_projekte_einfaches_linien_folgen.pdf

9. Appendix

Session 1:

R	B	G
255	203	152
255	203	152
255	203	152
255	203	152
255	203	152
255	203	152
255	203	152
255	203	152
255	203	152
255	203	152
255	203	152
Average	Average	Average
255	203	152

Table 1: Raw RGB calibration measurements for orange coloured paper

R	B	G
250	144	138
250	146	139
251	146	139
251	146	139
251	146	139
251	146	139

251	146	139
251	146	139
251	146	139
251	146	139
Average	Average	Average
251	146	139

Table 2: Raw RGB calibration measurements for red coloured paper

R	B	G
187	225	241
187	225	241
187	225	241
187	225	241
187	225	241
187	225	241
187	225	241
187	225	239
186	225	241
186	225	241
Average	Average	Average
187	225	241

Table 3: Raw RGB calibration measurements for blue coloured paper

R	B	G
258	258	260

258	258	260
258	258	260
258	258	260
258	258	260
258	258	260
258	258	260
258	258	260
258	258	260
258	258	260
Average	Average	Average
258	258	260

Table 4: Raw RGB calibration measurements for white coloured paper

R	B	G
188	231	216
188	231	216
188	231	216
188	231	216
188	231	217
188	231	217
188	231	216
188	231	216
188	231	217
188	231	217
Average	Average	Average

188	231	216.4

Table 5: Raw RGB calibration measurements for green coloured paper

R	B	G
259	239	239
259	239	239
259	239	239
259	239	239
259	239	239
259	239	239
259	239	239
259	239	239
259	239	239
259	239	239
Average	Average	Average
259	239	239

Table 6: Raw RGB calibration measurements for pink coloured paper

Session 2:

R	B	G
182	176	182
182	176	182
182	176	182
182	175	182
182	176	182
182	176	182
182	177	182
182	177	176
182	182	177
182	182	177
Average	Average	Average
182	177.3	180.4

Table 1: Raw RGB calibration measurements for orange coloured paper

R	B	G
180	133	169
180	133	169
180	133	169
180	133	169
180	133	169
180	133	169
180	135	171
181	135	171

180	135	171
181	135	171
Average	Average	Average
180	134	170

Table 2: Raw RGB calibration measurements for red coloured paper

R	B	G
183	233	271
183	233	271
183	233	271
183	233	271
183	233	271
183	233	271
125	202	255
125	202	255
125	202	255
125	202	255
Average	Average	Average
160	221	265

Table 3: Raw RGB calibration measurements for blue coloured paper

R	B	G
183	233	271
183	233	271
183	233	271

183	233	271
183	233	271
183	233	271
183	233	271
183	233	271
183	233	271
183	233	271
Average	Average	Average
183	233	271

Table 4: Raw RGB calibration measurements for white coloured paper

R	B	G
130	207	219
130	207	218
130	207	218
130	207	218
130	207	218
130	207	218
130	207	218
130	207	218
130	207	218
130	207	218
130	207	218
Average	Average	Average
130	207	218.1

Table 5: Raw RGB calibration measurements for green coloured paper

R	B	G
185	219	257
185	219	257
185	219	258
185	219	258
185	219	258
185	219	258
185	219	258
185	219	258
185	219	258
185	219	258
185	219	258
Average	Average	Average
185	219	257.8

Table 6: Raw RGB calibration measurements for pink coloured paper

Evaluation Day:

R	B	G
171	165	169
171	165	168
171	165	167
171	163	167
171	165	167
171	165	167
171	163	167
171	165	167
171	165	167
171	165	167
Average	Average	Average
171	164.6	167.3

Table 1: Raw RGB calibration measurements for orange coloured paper

R	B	G
177	128	155
177	128	155
177	128	155
177	128	155
177	128	155
177	129	156
177	128	155
177	128	156

177	128	155
177	128	155
Average	Average	Average
177	128	155

Table 2: Raw RGB calibration measurements for red coloured paper

R	B	G
110	178	235
110	178	233
109	178	233
109	178	233
109	178	233
109	178	233
109	178	233
109	178	233
109	178	233
109	178	233
109	178	233
Average	Average	Average
109	178	233

Table 3: Raw RGB calibration measurements for blue coloured paper

R	B	G
173	213	256
173	213	256
173	213	256

173	213	256
173	213	256
173	213	256
173	213	256
173	213	256
173	213	256
173	213	256
Average	Average	Average
173	213	256

Table 4: Raw RGB calibration measurements for white coloured paper

R	B	G
122	192	209
122	192	209
122	192	209
122	192	209
122	192	209
122	192	209
122	192	209
122	192	209
122	192	209
122	192	209
Average	Average	Average
122	192	209

Table 5: Raw RGB calibration measurements for green coloured paper

R	B	G
170	192	230
170	192	230
170	192	230
170	192	230
170	192	230
170	192	230
170	192	230
170	192	230
170	192	230
170	192	230
Average	Average	Average
170	192	230

Table 6: Raw RGB calibration measurements for pink coloured paper